Østfold University College

Prosjektrapport - TaskyFy

ITF20123-1 25V Rammeverk og .NET

Arshad Abba
Andreas Isaksen
Kjell-Magne Larsen
Marius Rørmark
Hedda Ørnelund Nilsen

May 27, 2025
Halden

Contents

List of Figures

Listings

List of Tables

Introduction

This report the final delivery in the course ITF20123-1 25V Rammeverk og .NET for a group consisting of five members. In this report all the work done in the project will be explained, and there is also a part which covers workshops and coding assigments in the course.

1.1 Group

The group in this project consists of five members:

- Andreas Isaksen
- Kjell-Magne Larsen
- Hedda Ørnelund Nilsen
- Marius Rørmark
- Arshad Abba

1.2 Overall project description

The goal of this project is to develop a reusable .NET library that can be distributed and integrated into other applications. The library should provide meaningful functionality that can assist developers in building modern, scalable, and maintainable applications.

Throughout the project, apply principles and best practices from the .NET ecosystem, including clean architecture, proper use of unit testing with MSTest, and structured source control using Git and Azure DevOps. Focus on writing high-quality code that is readable, testable, and easy to maintain.

The project will be developed in multiple stages:

- Release 1: A functional first version of the library with core functionality and tests.
- Release 2: An improved version based on feedback from user testing and extended features, while maintaining compatibility with the original API.

All development and planning will be done using Azure DevOps with branch policies, pull requests, and CI pipelines properly configured.

Methods

2.1 Chapter 2 - Framework Design Fundamentals

DO – TaskyFy is designed to support both common and advanced job scheduling scenarios. For the simplest use cases, a job only requires a trigger specifying when to execute and the code to run. This can be achieved either by manually instantiating a `Job` class or by using the reusable `JobBuilder` class. After defining the job, the user creates a `JobHandler` instance and calls the `StartOrAddJob` method to begin execution.

DO – For more advanced scenarios, users may define jobs that return values, use custom event triggers, or follow complex time intervals.

DO – To accommodate users with different skill levels, the `JobBuilder` class was introduced to guide users in creating jobs. It supports retry logic, priority settings, and return values. For basic usage, default values are used: normal priority, no retry logic, and no return values.

The `JobBuilder` is optional and serves as a learning and productivity aid. Jobs can be created without it if desired.

2.1.1 Principle of Low Barrier to Entry

DO – The main namespace contains only types for common scenarios. More advanced features are placed in the `Advanced` subnamespace. The total number of namespaces is kept to a minimum.

DO – Main classes provide constructors for both minimal and advanced usage. For example, the `JobHandler` class, which manages job scheduling, does not require arguments to instantiate. It also includes an overload that accepts an existing dictionary of jobs. Most logic is found in the `Job` class or delegated to trigger classes.

The simplest `Job` constructor only requires two arguments: a scheduling trigger and an `Action` delegate with user code.

DON'T – To create a scheduled job, users must instantiate a `JobHandler`, a `Job`, and a trigger. This goes against the principle of minimal setup. However, using `JobBuilder` allows for automatic creation of both the `Job` and trigger, reducing the number of objects users need to instantiate.

DON'T – Members intended for advanced usage are isolated in advanced types. The `Job` class is used for common scenarios, while the `AdvancedJob` class is reserved for more specialized use cases.

DO – Incorrect usage of the API is handled through exceptions, which signal misconfigurations or violations of expected usage patterns.

2.1.2 Principle of Self-Documenting Object Models

DO – All APIs are documented in both code and external references, including usage instructions for both basic and advanced scenarios.

DO – The documentation includes code snippets and full examples demonstrating how to construct and start jobs.

DO – Exceptions thrown by the API include meaningful and well-written messages that explain the error and suggest corrective action.

DO – All public APIs are strongly typed, providing clarity and minimizing errors at compile time.

2.2 Chapter 3 - Naming Guidelines

This section outlines how the naming guidelines from Chapter 3 of the Framework Design Guidelines have been applied in the TaskyFy scheduler library.

2.2.1 Capitalization Conventions

2.2.1.1 Capitalization Rules for Identifiers

DO - PascalCasing is used for all public type names, method names, property names, and event names.

Examples: JobHandler (class), StartJob (method in JobHandler), JobStatus (property in JobDefinition), ReadyToExecute (event in JobTrigger).

DO - camelCasing is used for parameter names.

Examples: jobStorageId (parameter in JobHandler.StartJob), cancellationToken (parameter in JobHandler.WorkerLoopAsy

DONT - Underscores are not used to differentiate words in identifiers. This is followed for all public identifiers.

2.2.1.2 Capitalizing Acronyms

DO - For two-letter acronyms, both letters are capitalized if not the first word of a camel-cased identifier.

DO - For acronyms of three or more characters, only the first letter is capitalized (unless it's the first word of a camel-cased identifier).

DONT - Any characters of an acronym are not capitalized at the beginning of a camel-cased identifier.

2.2.1.3 Case Sensitivity

DONT - The library does not rely on case alone to distinguish between names.

2.2.2 General Naming Conventions

2.2.2.1 Word Choice

DO - Identifier names are chosen to be easily readable.

Examples: JobHandler, JobStorage, TriggerType, JobDefinition.

DO - Readability is favored over brevity.

Example: GetAllActiveJobs (property in JobHandler) is preferred over a shorter, more cryptic name. WorkerLoopAsync is descriptive of its function.

DONT - Underscores, hyphens, or any other non-alphanumeric characters are not used in public identifiers.

DONT - Hungarian notation is not used.

2.2.2.2 Using Abbreviations and Acronyms

DONT - Abbreviations or contractions are not used as part of identifier names.

Example: Methods like GetJob are used instead of a hypothetical GetJb.

DONT - Acronyms that are not widely accepted are avoided.

2.2.2.3 Avoiding Language-Specific Names

DO - Semantic names are used rather than language-specific keywords for type names.

DO - Generic CLR type names are used if an identifier has no semantic meaning beyond its type.

Example: Parameters use names like value or item where appropriate, or more descriptive names like job or jobStorageId.

2.2.3 Names of Namespaces

DO - Namespace names are prefixed with a company/organization name.

Example: The root namespace is Hiof.TaskyFy, where "Hiof" represents the educational institution.

DO - PascalCasing is used for namespace components, separated by periods.

Example: Hiof.TaskyFy and Hiof.TaskyFy.Advanced.

DONT - The same name is not used for a namespace and a type within that namespace. This is adhered to.

2.2.4 Names of Classes, Structs, and Interfaces

DO - Classes and structs are named with nouns or noun phrases using PascalCasing.

Examples: JobHandler, JobStorage, JobDefinition, Job, TimedJobTrigger.

DO - Interfaces are named with adjective phrases, or occasionally nouns/noun phrases, and prefixed with the letter "I".

Example: IPublishJob<T> (used in JobTrigger.cs and JobHandler.cs).

CONSIDER - Derived class names sometimes end with the name of the base class.

Examples: TimedJobTrigger and EventJobTrigger derive from JobTrigger.

DO - When defining a class-interface pair where the class is a standard implementation, names differ only by the "I" prefix. This pattern is not explicitly used for custom pairs, but standard .NET pairs like IDisposable (implemented by JobHandler) are used.

2.2.4.1 Naming Enumerations

DO - A singular type name is used for an enumeration unless its values are bit fields.

Examples: JobStatus, TriggerType, JobPriority.

DO NOT - An "Enum" suffix is not used in enum type names. This is followed.

DO NOT - "Flag" or "Flags" suffixes are not used in enum type names (unless they are flags, which is not the case for JobStatus, TriggerType, JobPriority).

2.2.5 Names of Type Members

2.2.5.1 Names of Methods

DO - Method names are verbs or verb phrases.

Examples: StartJob, StopJob, AddJob, ExecuteJob (in Job.cs), BuildJob (in JobBuilder.cs), GetReferencedJob (in JobHandler.cs).

2.2.5.2 Names of Properties

DO - Properties are named using a noun, noun phrase, or adjective.

Examples: JobId, Description, JobStatus (in JobDefinition.cs), StartTime, Interval (in TimedJobTrigger.cs).

DO - Boolean properties are named with an affirmative phrase; optionally prefixed with "Is," "Can," or "Has" where it adds value.

Example: ContinuedRun (in EventJobTrigger, accessed via TriggerJobBuilder.cs) is an affirmative phrase.

2.2.5.3 Names of Events

DO - Events are named with a verb or verb phrase.

Example: ReadyToExecute (in JobTrigger.cs).

DO - Event handlers are named with the "EventHandler" suffix. The library uses the generic System.EventHandler<TEventAr (e.g., EventHandler<int> for ReadyToExecute), which follows this pattern.

DO - Two parameters named sender and e are used in event handlers.

The ExecuteJob method in JobTrigger.cs which raises the event takes (object? sender, int jobId). The HandleJobExecution method in JobHandler.cs which subscribes to this event has parameters (object? sender, int jobStorageId). Here, jobStorageId is used instead of a traditional EventArgs e parameter because the event is EventHandler<int>. This is a specific design choice for the event's data.

DO - Event argument classes are named with the "EventArgs" suffix.

2.2.5.4 Naming Fields

DO - PascalCasing is used in public static field names. Protected instance fields are not used. Public instance fields are not used, properties are used instead. The library follows this by not exposing public fields.

2.2.6 Naming Parameters

DO - camelCasing is used in parameter names.

Examples: jobStorageId (in JobHandler.StartJob), existingJobs (in JobHandler constructor), jobAction (in Job constructor).

DO - Descriptive parameter names are used.

Examples: description, triggerHandler, jobAction are clear about what they represent.

CONSIDER - Names based on a parameter's meaning rather than its type are used. This is generally followed.

2.3 Chapter 4 - Type design

2.3.1 DO Rules Followed

DO - Define only one type per file.
In the code, each class like `Job`, `AdvancedJob`, and `JobHandler` is placed in its own file. This makes the code easier to manage and understand..

DO - Use classes instead of structures for complex types.
Since the jobs and triggers need to hold state and logic, using classes like `Job` and `TimedJobTrigger` is the right choice.

DO - Use abstract classes when appropriate.
Some classes like `JobDefinition` and `JobTrigger` are made abstract and work as base classes for other, more specific job types.

DO - Define and implement a minimal interface.
For example, `IPublishJob<T>` only defines one event and does just what it needs to:

```
public interface IPublishJob<T> {
    event EventHandler<T> ReadyToExecute;
}
```

Listing 2.1: Minimal interface

DO - Use enums for sets of choices.
Enums like `JobPriority` and `JobStatus` clearly list valid choices. The code also checks that "None" values aren't used by mistake..

DO - Override `ToString()` on types that might appear in a UI.
Classes like `Job` and `AdvancedJob` give helpful text when printed, which makes debugging or displaying them easier.

DO - Provide a `Clone()` method if the object needs to be duplicated.
Some classes support deep copying so that a new copy of the job can be created safely:

```
public override JobDefinition Clone()
```

Listing 2.2: Cloning

2.3.2 DON'T Rules Avoided

DON'T - Make types inheritable without having a good reason.
Most classes are sealed unless they're meant to be extended. Inheritance is only used when it actually makes sense, like in `JobTrigger`.

DON'T - Use virtual members in constructors.
None of the constructors call virtual methods. This avoids issues with partially constructed objects.

DON'T - Override `Equals` without also overriding `GetHashCode`.
`JobDefinition` overrides both properly, so equality checks work as expected.

DON'T - Make fields public or protected.
Fields like `_jobFunc` are kept private and accessed with safe getters and setters:

```
private Func<TResult> _jobFunc;

public Func<TResult> JobFunc {
    get => _jobFunc;
    set => _jobFunc = value ?? throw new
       ArgumentNullException(nameof(value));
}
```

Listing 2.3: Making fields private

DON'T - Use mutable public static fields.
Static fields like loggers are private and not directly editable from outside the class.

DON'T - Use abstract classes unless they are intended for inheritance.
All abstract classes in this code are actually used by other classes, so they're there for a reason.

DON'T - Define operator overloads unless intuitive.
There are no operator overloads in the code, which avoids confusion.

DON'T - Implement `ICloneable`.
Instead of using the outdated `ICloneable`, the code uses clearly defined `Clone()` methods.

DON'T - Expose enum values named "None" unless they make sense.
Where enums have a "None" option, the code makes sure it isn't used incorrectly.

2.3.3 Type Naming and API Design

- Class names are clear and based on nouns like `Job` and `JobHandler`, and method names are verbs like `StartJob()`.
- The builder pattern is used to make configuration easier with method chaining.

- All public parts of the code have summaries written using XML comments.

2.4 Chapter 5 - member design

2.4.1 General member design guidlines

2.4.1.1 Member overloading

DO - try to use descriptive parameter names to indicate the default used by shorter overloads.

We wanted to change the name of continuedRun to keepListening to better reflect that the default value is false, normaly it doesn't continue to listen but if put true it will. We could change the name of this parameter that we explain more about later.

In another case where we had a parameter that would set the priority, we tried basing it on the default value, Naming the parameter normalPriority. This would imply that it's about choosing between "normal" and "not normal". This became confusing when there were more priorities than normal and not normal, so we decided to keep the original name JobPriority.

We have been discussing naming parameters like this, with some parameters getting more time than others.

DON'T - arbitrarily vary parameter names on overloads.

We consistently use the same parameter names across all overloads when the semantics are the same. There is no arbitrary variation in naming.

AVOID - being inconsistent in ordering parameters for overloaded members.

We do have one exception with the constructors for AdvancedJob and Job. The overloads follow the parameter order defined within their own classes but differ from the order in the parent class constructor. In this case, we chose to prioritize consistency within the class, even though it introduces inconsistency with the base class.

Apart from this, all other overloads follow a consistent parameter order.

DO - make only the longest overload virtual.

We currently do not have any virtual overloads, so this guideline does not apply.

DON'T - use ref, out, or modifiers to overload members.

We do not use ref, out, or in any of our overloaded methods. Out and in are used but in other contexts.

DON'T - have overloads with parameters at the same position and similar types, yet with different semantics.

We previously had one internal overload where parameters of the same type appeared in the same position but carried different meanings. This also resulted in inconsistent ordering parameters. To fix this, we reordered the parameters.

Shorter constructor:

```
public Job(string description, JobTrigger triggerHandler,
   Action jobAction, JobPriority jobPriority)
```

Listing 2.4: Shorter constructor

Before:

```
internal Job(string jobId, string description, JobTrigger
   triggerHandler, Action jobAction, JobPriority jobPriority)
```

Listing 2.5: Before change of constructor

After:

```
internal Job(string description, JobTrigger triggerHandler,
   Action jobAction, JobPriority jobPriority, string jobId)
```

Listing 2.6: After change of constructor

Although the revised constructor is internal, the change helps prevent misplacement of parameters (especially in dynamically typed languages) and improves clarity.

DO -allow null to be passed for optional arguments.

We don't have optional parameters that are reference types, so this rule does not currently apply.

DON'T - have overloads with a generic type parameter in one method and a specific type in another.

We do not use generic types in our overloaded methods.

CONSIDER - using default parameters on the longest overload of a method.

Default parameter is done on most advanced overload in EventJobTrigger constructor where two extra parameters have default values.

DO - provide a simple overload, with no default parameters, for any method with two or more defaulted parameters. DO

For EventTrigger, we follow this practice by providing both a simple constructor and a more advanced version with default parameters:

Example:

```
public EventJobTrigger(IPublishJob<int> eventPublisher)
```

Listing 2.7: Simple Eventrigger constructor

```
public EventJobTrigger(IPublishJob<int> eventPublisher, int
    triggerThreshold = 1, bool keepListening = false)
```

Listing 2.8: Advanced Eventrigger constructor

This approach helps users understand that they can rely on default values without having to specify them explicitly.

DON'T - use default parameters for any parameter type other than CancellationToken, on interface methods or virtual methods on classes.

None of our interface or virtual methods use default parameters.

DON'T - change the default value for a parameter once it has been publicly released.

All default parameter values were established early in development and have remained unchanged since the first release.

DON'T - have two overloads of a method with "compatible" required parameters that both use default parameters.

We don't have two overloads with "compatible" default parameters.

AVOID - having two overloads of the same method that both use default parameters.

We don't have two overloads that both use default parameters.

DON'T - DO NOT have different defaults for the same parameter in two overloads of the same method.

We only have a constructor with multiple default parameters but they are both on the same overload, so it is still clear what overload is used.

DO - move all default parameters to the new, longer overload when adding optional parameters to an existing method.

So far, we haven't needed to extend existing methods with new optional parameters. Therefore, we haven't needed to apply this technique yet. However, this might become relevant in future versions—for example, if we expand setEventTriggerThreshold to support more complex scheduling options (e.g., different thresholds for weekdays vs. weekends).

2.4.1.2 implementing interface members explicitly

AVOID implementing interface members explicitly without having a strong reason to do so

In our framework, we've only defined one interface and have relied on a minimal number of .NET interfaces. We haven't encountered situations where multiple interfaces define members with the same name, so we've had no compelling reason to implement interface members explicitly.

CONSIDER - implementing interface members explicitly if the members are intended to be called only through the interface.

Our only interface member is the event ReadyToExecute. While it could be implemented explicitly, we chose not to. This event is designed to be flexible—it's meant to allow users to implement it on any class they want the EventTrigger to interact with. Additionally, we wanted the flexibility to pass more than just an int as event data. As a result, ReadyToExecute is not exclusively called through the interface.

CONSIDER - implementing interface members explicitly to simulate variance.

We haven't encountered a scenario where we needed to use explicit implementation to simulate variance. So far, this has not been necessary for our framework.

CONSIDER - implementing interface members explicitly to hide a member and add an equivalent member with a better name.

In our case, we have not found value in using this technique. While it can be useful in situations where renaming a member is necessary, we've prioritized choosing good names upfront to avoid the need for this kind of workaround later.

For example, we wanted to rename the parameter continuedRun to something more descriptive, reflecting that its default is false. However, renaming the parameter would also mean that we need to rename our members and properties to adhere to other guidelines. This would break backward compatibility. Although the name could be improved, it's still reasonably descriptive. We concluded that using an explicit interface member to "rename" it would introduce more confusion than it resolves.

This strategy might become more useful as the framework evolves, but for now, it doesn't provide clear benefits.

DON'T - use explicit members as a security boundry

We do not use explicit interface members for security purposes in our framework.

DO - provide a protected virtual member that offers the same functionality as the explicitly implemented member if the functionality is meant to be specialized by derived classes.

This situation is not currently relevant to our framework.

2.4.1.3 choosing between properties and method

DO - use property, rather than a method, if the value of the property is stored in the process memory and the property would just provide access to the value.

This is the most common reason we've chosen properties over methods in our framework. Many of our properties are straightforward accessors for values stored in memory. For example, JobId in JobDefinition is implemented as a property because it simply exposes the identifier of the job.

We apply this principle consistently wherever members are just returning values from fields without performing any computation or logic.

DO - use a method, rather than a property, in the following situations.

- The operation is orders of magnitude slower than a field access would be.
  - We've taken care to avoid using properties for potentially slow operations, particularly in the Logger class, which interacts with the file system. Using methods in this context helps make performance considerations more explicit.
- The operation is a conversion.
  - We follow this rule for conversions. For instance, we use the ToString() method (as is conventional), and we don't have any conversion logic implemented as properties.
- The operation returns a different result each time it is called, even if the parameter doesn't change.
  - We avoid this pattern in our properties. Although we use Guid, it's only generated in the constructor and stored as a field. The corresponding property has a getter that simply returns this stored value, ensuring consistency.
- The operation has a significant and observable side effect.
  - We don't have any Properties with known observable side effects.
- The operation returns a copy of an internal state
  - For copies we use methods to be sure that we follow this guidline.
- The operation returns an array
  - We have not used arrays in our framework, so this case is not applicable.

2.4.2 Property design

DO - create get-only properties if the caller is not able to change the value of the property.

Our framework includes many get-only properties, such as JobId in the JobDefinition class. This property is intended to be immutable and represent a unique identifier that should not be changed after initialization.

There are a few cases where a get-only property would have been more appropriate, but due to decisions made in version 1 of the framework, those properties were released with both getters and setters. One such example is continuedRun, which ideally should have been read-only. Its value is meant to be updated via dedicated methods, not by direct assignment.

DON'T - provide set-only properties or properties with the setter having broader accessibility than the getter.

We adhere to this guideline strictly. Our framework does not include any set-only properties, and we do not allow setters to have broader accessibility than their corresponding getters. In some cases, the getter is more accessible than the setter, which aligns with the recommended design.

DO - provide sensible default values for all properties, ensuring that the defualts do not result in a security hole or terribly inefficient code.

The only property in our framework that currently has a default value is TriggerThreshold, which is part of the EventTrigger class. It defaults to 1, meaning the job will execute on the first signal received.

We considered setting the default to 0, which would effectively mean the job triggers on every signal. However, this can be ambiguous as some users may interpret 0 as “immediate,” while others might expect 1 to mean "trigger after receiving one signal." Setting the default to 1 provided a clearer and more intuitive behavior, especially when configuring higher thresholds (e.g., 2 means execute on the second signal).

To the best of our knowledge, this default value does not introduce any security vulnerabilities or inefficiencies.

DO - allow properties to be set in any order, even if this results in a temporary invalid state of the object.

None of our properties are interdependent to the extent that setting them in a particular order would cause temporary invalid states. Each property can be set independently without affecting object validity.

DO - preserve the previous value if a property setter throws an exception.

For properties that may throw exceptions during assignment, we ensure that the new value is only set after validation succeeds. If an exception is thrown, the property retains its previous value, thereby maintaining a consistent and predictable state.

AVOID - throwing exceptions from property getters.

We have no properties where the getter throws an exception.

2.4.2.1 Indexed property design

We dont use indexed properties in our framework. So, this subchapter is not relevant to our framework.

2.4.2.2 Property change notifications events

We don’t use events to notify a change in any of our properties, the event we are using is meant for when the triggers conditions are met to execute a Job. So, this subchapter is not relevant to our framework.

2.4.3 Constructor design

CONSIDER - providing simple, ideally default, constructors.

We provide default constructors for the JobBuilder and JobHandler classes. In particular, JobBuilder relies on method chaining to construct jobs, and providing an easy entry point was a key design goal. Most of our other classes also include simple constructors, with the notable exception of EventJobTrigger, which requires a publisher for initialization. However, this is abstracted through the builder pattern, so users don't need to instantiate EventTrigger manually.

CONSIDER - CONSIDER using a static factory method instead of a constructor if the semantics of the desired operation do not map directly to the construction of a new instance, or if following the constructor design guidlines feels natural.

We opted for the builder pattern rather than factory methods when constructing jobs. This decision was based on the desire to offer a more intuitive and fluent experience for users building Job instances.

DO - use constructor parameters as shortcuts for setting main properties.

This principle is followed throughout the framework. For example, EventJobTrigger can be initialized using either a default constructor followed by property assignment, or by using a constructor that takes the publisher directly. Both approaches yield the same result, giving the user flexibility.

DO - use the same name for constructor parameters and property if the constructor parameters are used just to set the property.

We've maintained consistent naming between constructor parameters and the properties they set. One example is the continuedRun parameter, which sets the ContinuedRun property. While we considered renaming the parameter to keepListening (which we believe better conveys its purpose and default value), doing so would require renaming the property as well to maintain consistency. This change would break compatibility with version 1, so we have chosen not to make the change.

CONSIDER - CONSIDER adding a property for each parameter in a constructor.

In most cases, we expose constructor parameters through corresponding properties. One exception is JobHandler, which accepts a KeyValuePair in its constructor to support reloading persisted jobs. This parameter doesn't have a direct property, but its data can still be accessed via the GetAllJobs method, which returns a copy of the job collection.

DO - minimal work in the constructor.

Our constructors generally avoid performing significant work. The most substantial logic appears in the internal constructor for JobStorage, shown below:

```
internal JobStorage(IEnumerable<KeyValuePair<int,
    JobDefinition>> existingJobs)
{
     if (existingJobs is null)
```

```
    {
        throw new ArgumentNullException(nameof(existingJobs),
           "Existing jobs cannot be null");
    }

    _jobs = new ConcurrentDictionary<int,
       JobDefinition>(existingJobs);

    SyncJobIdHelpers();
}
```

Listing 2.9: Minimal work in constructor

Even here, the logic is minimal and focused on essential validation and setup.

DO - throw exceptions form instance constructors, if appropriate.

Most of our constructors throw exceptions based on validation. Here is an example where our constructor throws an exception.

```
public Job(string description, JobTrigger triggerHandler,
   Action jobAction)
    : base(description, triggerHandler)
    {
    ArgumentNullException.ThrowIfNull(jobAction,
       nameof(jobAction));

    _jobAction = jobAction;
    }
}
```

Listing 2.10: Constructor with exception

DO - explicitly declare the public default constructor in classes, if such a constructor is required.

All public classes that require default constructors explicitly declare them.

AVOID - explicitly defining default constructors on structs

We do not use structs in our framework, so this guideline does not apply.

AVOID - calling virtual members on an object inside its constructor

We have only one virtual member in our framework, and it is not invoked within its constructor, thus adhering to this guideline.

2.4.3.1 Type constructor guidelines

DO - make static constructors private.

We have one static constructor in the Logging class, which has been marked as private in accordance with this guideline.

DON'T - throw exceptions form static constructors.

Our static constructor in the Logging class does not throw any exceptions.

CONSIDER - CONSIDER initializing static fields inline rather than explicitly using static constructors, because the runtime is able to optimize the performance of types that don't have an explicitly defined static constructor.

We've followed this guideline by initializing the static fields timeStamp and logFilePath inline in the Logging class, avoiding the use of a static constructor for these assignments.

DO - use the term "raise" for events rather than "fire" or "trigger".

Initially, we did not follow this guideline, and our event naming used terms like "trigger". This was not recognized until after the release of version 1. Since then, we've corrected this terminology in XML documentation and other descriptive texts, though we have not renamed the actual event identifiers to avoid breaking existing code.

DO - use System.EventHandler<T> instead of manually creating new delegates to be used as event handlers.

All events in our framework utilize the EventHandler delegate rather than custom delegate types.

CONSIDER - using a subclass of EventArgs as the event argument, unless you are absolutely sure the event will never need to carry any data to the event handling method, in wich case you can use the EventArgs type directly.

At present, our event only needs to send an int value. However, the event is already structured to support the use of a subclass of EventArgs if the need arises to transmit more complex data in the future.

DO - use protected virtual method to raise each event.

We currently use a method to raise the ReadyToExecute event in the JobTrigger class. However, this method is public rather than protected virtual. In hindsight, it should have followed the guideline to allow safe overriding in derived classes. We have avoided changing this post-release to prevent breaking user code.

DO - take one parameter to the protected method that raises an event.

We currently violate this by using a public method with the signature:

```
public void ExecuteJob(object? sender, int jobId)
    => ReadyToExecute?.Invoke(sender, jobId);
```

Listing 2.11: Raising an event

Ideally, it should be rewritten as:

```
protected virtual void OnExecuteJob(int jobId)
    => ReadyToExecute?.Invoke(this, jobId);
```

Listing 2.12: Raising an event correctly

This would prevent users from passing an incorrect sender and would better align with standard event-raising practices.

DON'T - pass null as the sender when raising a nonstatic event.

We do not pass null directly as the sender in our code. However, due to the incorrect access modifier on the event-raising method, there is a risk that users may raise the event with a null sender, which violates this guideline.

DO - pass null as the sender when raising a static event.

We currently do not have any static events, so this does not apply to our framework.

DON'T - pass null as the event data parameter when raising an event.

Our intent is not to pass null as the event data. However, since the event-raising method is publicly accessible, users could inadvertently pass null, which could lead to unintended behavior.

CONSIDER - raising events that the end user can cancel.

This has not yet been implemented, but it is a feature we recognize as potentially valuable. For example, giving users the ability to cancel events could support scenarios like pausing event execution to perform critical operations such as backing up the system before a shutdown or hardware update.

2.4.4 Field design

DON'T - provide instance fields that are public or protected.

All our instance fields are private.

DO - use constant fields for constants that will never change

We currently do not have any constants defined in our framework.

CONSIDER - using public static readonly fields for predefined object instances.

DON'T - Our framework does not define any predefined object instances, so this guideline has not been applicable.

DON'T - assign instances of mutable types to public or protected readonly fields

All mutable types in our framework are either private or internal, and none are assigned to public or protected readonly fields.

2.4.5 Extension Methods

AVOID - frivolously defining extension methods.

We don't use any extension methods in our framework.

CONSIDER - using extension methods to provide helper functionality relevant to every implementation of an interface.

We have not encountered any need for helper methods applicable to every implementation of our interfaces.

CONSIDER - using extension methods when an instance method would introduce an unwanted dependency.

There have been no scenarios in our project where adding an instance method would introduce an unwanted dependency.

CONSIDER - using generic extension methods when an instance method on a generic type is not well defined for all possible type parameters.

Our use of generics is limited—primarily to event handling. While generic extension methods might offer some future utility, they have not been necessary so far.

DO - throw an ArgumentNullException when the this parameter in an extension method is null.

Since we haven't implemented any extension methods, this guideline does not currently apply.

AVOID - defining extension methods on Systme.Object.

We have not defined any extension methods in our framework.

CONSIDER - naming the type that holds extension methods for its functionality.

We haven't used any extension methods in our framework.

DON'T - put extension methods in the same namspace as the extended type.

We haven't used any extension methods in our framework.

AVOID - defining two or more extension methods with the same signature.

We haven't used any extension methods in our framework.

2.4.6 Operator overloads

We don't use operator overloads in our framework

2.4.7 Parameter design

DO - use the least derived parameter type that provides the functionality required by the member.

We have consistently used the least derived type necessary to fulfill each member's functionality. In most cases, we rely on primitive types. For collections, we use abstractions such as IEnumerable, which provide the required iteration capabilities.

DON'T - use reserved parameters.

Our framework does not use any reserved parameters. Instead, we introduced method overloads to expand functionality between releases.

DON'T - have publicly exposed methods that take pointers, arrays of pointers, or multidimensional array as parameters.

We do not use pointers or multidimensional arrays in any method parameters within our framework.

DO - place all out parameters following all of the by-value and ref parameters, even if it results in an inconsistency in parameter ordering between overloads.

We follow these guidelines consistently. All out parameters appear at the end of the parameter list.

Example:

```
public bool TryGetJob(int jobStorageId, out JobDefinition?
    job)
```

Listing 2.13: Out parameters position

DO -be consistent in naming parameters when overriding members or implementing interface members

We have been consistent with naming parameters on overriding.

Example :

```
fra JobTrigger

public abstract void Start(int jobId);

Fra TimedJobTrigger

public override void Start(int jobId)
```

Listing 2.14: Parameter overloading

2.4.7.1 Choosing between enum and boolean parameters

DO - use enums if a member would otherwise have two or more boolean parameters.

We currently have no methods that require multiple booleans.

DON'T - use boolean parameters unless you are absolutely sure ther will never be a need for more than two values.

We could have used a boolean to determine if the trigger is timed trigger or event trigger, but we were pretty sure we would at some point make more trigger types and that potentially have options for the user to make custom triggers. The enum in this case is also much more readable than just having a boolean expression.

In the EventJobTrigger we were quite sure that the choose between continuously running and not would either be true or false, and since the default value is false it is readable.

We have some more cases like this, for instance isScheduled we were pretty confident would never need more than two values. We ended up not using isScheduled but it depicts how we have been thinking about this guideline.

CONSIDER - using booleans for constructor parameters that are truly two-state values and are simply used to initialize boolean properties.

An example of this is in the EventJobTrigger class, where a bool constructor parameter is used to initialize the ContinuedRun property..

2.4.7.2 Validating arguments

DO - validate arguments passed to public, protected, or explicitly implemented members.

Most argument validation in our framework happens immediately upon method call. However, in some cases (e.g., EventJobTrigger), validation is deferred until a related method is invoked.

Example: Validation of the Publisher property occurs during the Subscribe and Unsubscribe methods, not at construction or property assignment time.

DO - throw ArgumentNullException if a null argument is passed and the member does not support null arguments.

We throw ArgumentNullException in all cases where null is not an acceptable argument.

DO - validate enum parameter.

We did initially validate the TriggerType enum since it had to be checked what value it had annyways but we forgot in our haste the JobPriority enum and had to go back and add validation.

DON'T - use Enum.IsDefined for enum range checks.

We follow this guideline and avoid using Enum.IsDefined for range validation.

DO - be aware that mutable arguments might have changed after they were validated

To guard against changes after validation, we encapsulate mutable structures. For instance, ConcurrentDictionary<int, JobDefinition> is private, and we populate it using an IEnumerable<KeyValuePair<int, JobDefinition», which is immutable. Additionally, we validate this data upon setting.

2.4.7.3 Parameter passing

AVOID - using out or ref parameters except when implementing patterns that require them.

We have been using try pattern and have therefore used out parameters in some parts of our framework.

DON'T - pass reference types by reference.

We have not been using ref, only out.

2.4.7.4 Members with variable number of parameters

We do not utilize of variable parameters.

2.4.7.5 Pointer parameters

We don't use pointers in our framework

2.4.7.6 Using tuples in member signatures

We don't use tuples in our framework

2.5 Chapter 6 - Designing for extensibility

2.5.1 Extensibility Mechanisms

2.5.1.1 Unsealed classes

CONSIDER – All public classes are unsealed and can accept further inheritance and customization from users.

2.5.1.2 Events and callbacks

CONSIDER – A core part of the library is making users able to provide custom code through callbacks in the Job class.

CONSIDER – EventTrigger implements IPublish interface, shown in figure 2.1 which has an event that holds another class implementing the interface, which makes it possible for users to provide custom conditional code to trigger when events should fire code in callbacks. Users can also create custom triggers for extended usage.

```
public interface IPublishJob<T>
{
    /// <summary>
    /// The event that fires when a job is ready to be executed.
    /// </summary>
    event EventHandler<T> ReadyToExecute;
}
}
```

Figure 2.1: IpublishJob<T> interface must be implemented on classes that should trigger a job to execute code provided by user

DO – For normal usage, Job class lets users provide custom code with no return values using Action delegate. Func delegate is implemented in AdvancedJob class, making it more powerful for users wanting to return values from a scheduled task. Keep in mind that no callbacks allow arguments to be passed.

DO – The team accepts the security risks by letting users provide custom code by calling delegates such as Action and Func in Job and AdvancedJob classes.

2.5.1.3 Virtual members

DONT – There are virtual methods in JobDefenition base class, one being the Result property, shown in figure 2.2. If overridden in inherited class, it will return null, which is due to normal Job classes does not returning a result, while AdvancedJob does return a value. Further development or custom implementations sparks the possibility of creating new types of Jobs that can return even more values. The reason for keeping the method public is to signal users can implement their own Job classes.

```
public interface IPublishJob<T>
{
    /// <summary>
    /// The event that fires when a job is ready to be executed.
    /// </summary>
    event EventHandler<T> ReadyToExecute;
}
}
```

Figure 2.2: Virtual property Result in Jobdefenition, returning null if not overriden by child.

The other virtual method is IsExceptionTransient, which throws exceptions in some cases should not lead to a retry, such as OpertionCanceledException, NullReferenceException and ArgumentException. Making IsExceptionTransient virtual gives more flexibility to concrete implementations on which exceptions that are temporarily for certain type of work the job is doing. For example, a job communicating with a database can override IsExceptionTransient to recognize certain SQL errors that should be temporary, something that the base class wouldn't normally know about.

2.5.1.4 Abstractions

DONT – Abstractions are only implemented in base classes such as JobDefenition and JobDefenitionBuilder which has concrete implementations in Job, AdvandedJob and JobBuilder classes.

DO – Abstract class or interface have carefully been chosen when doing abstractions. For classes that might be expanding and have multiple fields and properties abstract have always been preferred, like in JobBuilder. Interface has been chosen for specialized tasks with single responsibilities that users might want to extend, like IPublishJob to provide custom code for triggers to subscribe and respond to. Preferring abstract classes makes the library easy to expand upon in the future with new and improved features.

2.5.2 Base classes

CONSIDER – Base classes such as JobDefenition and JobDefenitionBuilder are placed in a separate namespace Advanced.

AVOID – Base classes do not inherit bad naming practices like "Base".

2.5.3 Sealing

DONT – No classes are sealed.

2.6 Chapter 7 - Exceptions

2.6.1 Exception Throwing

DONT - No methods are returning error codes. However, one method RemoveJob shown in figure 2.3 in JobHandler is returning a bool if the removal was successful, otherwise it throws an exception which is either called by the method itself or returned from an internal call from JobStorage.

```
/// <summary>
/// Removes a job from the <see cref="JobStorage"/>.
/// </summary>
/// <param name="jobId">Identifier on job to be removed.</param>
/// <returns>
/// <c>true</c> if the job was successfully removed; otherwise, <c>false</c>.
/// </returns>
5 references
public bool RemoveJob(int jobStorageId)
{
    if (_activeJobs.TryGetValue(jobStorageId, out var existingJob))
    {
        if (existingJob.JobStatus == JobStatus.Active)
        {
            logger.LogError("Cannot update an active job");
            throw new InvalidOperationException("Cannot remove an active job. Stop the job before removing it.");
        }
        StopJob(jobStorageId);
        logger.LogInformation($"Stopped job with ID: {jobStorageId}");
    }

    return _jobStorage.RemoveJob(jobStorageId);
}
```

Figure 2.3: RemoveJob method in JobHandler returns a bool if successfully removed a job. Throw an exception otherwise.

DO – Most members that cannot fulfil their task that should be interactable by users will throw exceptions, mostly ArgumentOutOfRange, InvalidOperation, ArgumentNullException and KeyNotFoundException. ExecuteJob in Job and AdvancedJob class is an unintended exception that was not meant to be public. Shown in figure 2.4, described in detail in CHAPTER XX.

```
/// <summary>
/// Executes the job function and returns the result.
/// </summary>
/// <returns>The result of the job function.</returns>
2 references
public override void ExecuteJob() => _jobAction();
```

Figure 2.4: ExecuteJob in Job class should have been internal instead of public.

DONT – When it comes to normal flow of control, only members that can be passed arguments will generally throw exceptions if it leaves the property or method in a state that cannot fulfil their task. Most members in JobHandler class throw exceptions if the desired Job to add or search for is returning a failing result. Described in more detail in chapter XX, GetJob method follows the try pattern where users can search for and return no results without getting a KeyNotFoundException. Some participants in the user test found themselves getting a lot of KeyNotFoundExceptions when passing the wrong JobStorageId to StartJob method in JobHandler. The solution, shown in figure 2.5, was to introduce a new method StartOrAddJob where a referenced JobDefenition or derived types can be passed instead to limit the number of exceptions and method calls to start scheduling jobs.

```
public int StartOrAddJob(JobDefinition job)
{
    foreach (var jobKV in _jobStorage.Jobs)
    {
        if (job == jobKV.Value)
        {
            StartJob(jobKV.Key);
            return jobKV.Key;
        }
    }

    _jobStorage.AddJob(job);
    int addedJobId = _jobStorage.GetJobIdCounter(job);
    StartJob(addedJobId);
    return addedJobId;
}
```

Figure 2.5: StartOrAddJob is a more flexible API that combines the logic of Add and StartJob. It returns the storageID made by JobHandler, which did not include AddJob and StartJob.

DO – All publicly callable members with parameters will throw exceptions if violating the member contract.

DONT – There are no members that can or cannot throw based on some parameter option.

2.6.2 Choosing the right exceptions

DONT – No custom exceptions are made to purely communicate usage errors, only standard common exceptions.

CONSIDER – One custom exception NumberOfRetriesException, shown in figure 2.6, is made to stop scheduled jobs from executing bad code implemented by user. This exception is only thrown if the user has flagged a job to only retry a couple of timed before stopping. Otherwise, an interval-based time job could keep going forever, producing bad results.

```
6 references
public class NumberOfRetriesException : Exception
{
    1 reference
    public int MaxRetries { get; }

    2 references
    public int AttemptsMade { get; }

    0 references
    public NumberOfRetriesException() : base() { }

    0 references
    public NumberOfRetriesException(string message) : base(message) { }

    0 references
    public NumberOfRetriesException(string message, Exception innerException) : base(message, innerException) { }

    1 reference
    public NumberOfRetriesException(string message, int maxRetries, int attemptsMade, Exception? innerException)
        : base(message, innerException)
    {
        MaxRetries = maxRetries;
        AttemptsMade = attemptsMade;
    }
}
```

Figure 2.6: NumberOfRetriesException is derived from Exception base class.

DO – NumberOfRetriesException was created due to not finding any suitable standard exceptions to communicate this type of error.

2.6.2.1 Error message design

DO – All exceptions, except null throwing, do have meaningful and clear error messages specifying what went wrong and how to avoid it.

DO – Error messages are written grammatically correctly as possible.

AVOID – No question or exclamation points are used in any exception messages.

DONT – No security sensitive information is exposed in exception messages, only arguments input by users, like invalid jobStorage Id.

2.6.2.2 Exception handling

DONT – Since code provided by the user can lead to a lot of different exceptions, NumberOfRetriesException was created to not swallow errors by catching nonspecific exceptions in System.Exception and so on.

2.6.3 Standard exception types

DONT – The library does not throw or catch SystemException or any forms for ApplicationException. It is catching a custom exception NumberOfRetries, shown in figure 2.7 that is derived from Exeption class.

DO – InvalidOperationException is thrown in any place where a property setter or method call would be left in an inappropriate state. Figure 2.7 shows an example where InvalidOperationException is thrown when BuildJob is chained at the end of a builder to return a new Job, but no trigger is configured. That would lead to an incomplete Job that is not able to perform a scheduled task.

```
/// <summary>
/// Builds a <see cref="Job"/> with the specified parameters.
/// </summary>
/// <returns>A new <see cref="Job"/> instance specified builder settings.</returns>
/// <exception cref="InvalidOperationException">Throws if the <see cref="BuildJob"/> is called bef
5 references
public override JobDefinition BuildJob()
{
    return _triggerType switch
    {
        TriggerType.TimedTrigger => new Job(Description, _timedTrigger, JobAction, JobPriority),
        TriggerType.EventTrigger => new Job(Description, _eventTrigger, JobAction, JobPriority),
        _ => throw new InvalidOperationException("No trigger has been configured, it is needed to
    };
}
```

Figure 2.7: BuildJob in JobBuilder throws InvalidOperationException if no trigger is configured, leaving creation of a job in an invalid state.

DO – ArgumentException is thrown from any members where users can pass technically correct arguments but may be inappropriate depending on the usage. For example: ArgumentException will be thrown from AddJob if the Job with the same ID already exists in a Dictionary. It is generally used when no derived or more concrete exception is appropriate to use. ArgumentOutOfRangeException is usually thrown where arguments such as only positive integers are allowed, but users can mistakenly pass a negative value. ArgumentNullException is thrown from constructors, property setters and methods where null is not allowed.

DO – The paramName property alongside the use of value is present in all throws from ArgumentExceptions and its derived types where possible.

DONT – The library is not throwing or catching any NullReference, IndexOutOfRange, StackOverflow, OutOfMemory, Com, SEH and ExecutionEngineException.

DO – Although WorkerLoopAsync method in JobHandler is private, it's the only method which can throw OperationCanceledException from Task.Delay method.

2.6.4 Designing custom exceptions

DO – NumberOfRetriesException is derived from base System.Exception class.

AVOID – No deep exception hierarchies are created.

DO – Custom exceptions do have "Exception" as a naming suffix.

DO – Custom exception is made serializable at runtime by implementing ISerializable interface.

DO – Common constructors are provided for custom exceptions.

2.6.5 Exceptions and performance

CONSIDER – The try pattern is used by TryGetJob and GetJob, shown in figure 2.8 method in JobHandler, which calls the underlying internal method in JobStorage, assumes users would use it a lot for retrieving existing jobs in the field. It was considered to add more try methods like TryAdd, TryStart or TryStopJob, but was rejected due to many similar methods that could lead to confusion in on where to apply which methods. Try methods did not receive any feedback during user testing release 1.0.0, leaving more try methods out of development.

```
public bool TryGetJob(int jobStorageId, out JobDefinition? job)
{
    logger.LogInformation($"Attempting to get job with ID: {jobStorageId}");
    return _jobStorage.TryGetJob(jobStorageId, out job);
}
```

Figure 2.8: TryGetJob methods follows the Try-pattern, calling underlying logic in JobStorage.

DO – The only custom med try method, TryGetJob, shown in figure 2.8, does prefix "try" and returns a bool to implement the try pattern.

DO – TryGetJob has its underlying logic in an internal method with the same name in JobStorage, shown in figure 2.9. It returns false when passing a JobStorageId that does not correspond to any added jobs in the system. It calls an underlying TryGetValue method from Concurrent Dictionary, which is responsible for throwing any other exceptions when allowed arguments in range are passed.

```
internal bool TryGetJob(int jobStorageId, out JobDefinition? job)
{
    if (jobStorageId < 1)
    {
        logger.LogError("Job id must be greater than 0.");
        throw new ArgumentOutOfRangeException(nameof(jobStorageId), "Job id must be greater than 0.");
    }
    if (_jobs.TryGetValue(jobStorageId, out job))
    {
        job = job.Clone();
        return true;
    }

    job = null;
    return false;
}
```

Figure 2.9: TryGetJob in JobStorage class holds the underlying logic for retrieving jobs based on storageId.

DO – TryGetJob does have a similar GetJob method which throws KeyNotFoundException if a jobStorageId does not correspond to any added jobs in the system.

DO – TryGetJob does successfully returns a value as an out parameter. If no corresponding jobs are found, the out parameter is null.

DO – "Write default(T) to the out parameter from Try method that returns false" (Cwalina and Abrams, 2009, see p. 284), not sure if it's done correctly because the rule does not say when to return default if

applying to both, or either value and reference types. Ended up following TryGetValue documentation under remarks, stating reference types can return null (Microsoft, 2025a).

2.7 Chapter 8 - Usage guidelines

2.7.1 Arrays and Collections

No arrays or collections are used by or returned by any public or private members, only dictionaries.

DONT – The library is only returning strongly typed dictionaries such as IReadOnlyDictionary<int, JobDefinition> GetAllActiveJobs in JobHandler class.

DONT – No ArrayList or List is used in public APIs.

DONT – No hashtables or Dictionaries are used in public APIs, only IReadOnlyDictionaries.

DONT – No usage of IEnumerator<T> or IEnumerator or any other type that implements these interfaces.

DO – JobHandler, or the underlying JobStorage has a ConcurrentDictionary to hold all jobs. Users can import existing jobs by calling JobHandler constructor taking the least-specialized type possible, IEnumerable<KeyValuePair<TKey, TValue».

Users must call GetAllJobs or GetAllActiveJobs in JobHandler to retrieve all added jobs, which returns an IReadOnlyDictionary. During release 1.0.0, JobHandler only had one overload constructor to pass IDictionary of existing jobs as an argument, making it impossible to export jobs with GetAllJobs to a new instance of JobHandler. Due to naming confusion on IReadOnlyDictionary and IDictionary, no one noticed that they did not work together (Microsoft, 2025b). An overload constructor in JobHandler fixed it, shown in figure 2.10. Also changing the parameter in JobStorage, making it possible to export and jobs in release 2.0.0 with no breaking changes.

```
public JobHandler(IEnumerable<KeyValuePair<int, JobDefinition>> existingJobs)
{
    _jobStorage = new JobStorage(existingJobs);
}
```

Figure 2.10: New overload in JobHandler constructor accepts a variety of dictionaries with existing jobs.

DONT – Public dictionary properties are get only and do not offer setters. Only set a dictionary through a new instance of JobHandler with the constructor.

DO – Consider this rule apply to dictionaries as well, IReadOnlyDictionary is used to return a dictionary of all jobs to users, representing read-only dictionaries.

DONT – Dictionary properties in JobHandler such as GetAllJobs returns an empty IReadOnlyDictionary instead of null.

2.7.2 DateTime and DateTimeOffset

DO – Triggers and logging for creation of Jobs uses DateTime rather DateTimeOffset because it was not found necessary to know the exact time zone in most cases.

DO – While StartTime and EndTime properties in TimedJobTrigger uses DateTime object, Interval benefits more from using TimeSpan.

2.7.3 ICloneable

CONSIDER – Clone methods are implemented to perform deep-copies of Job and AdvandedJob classes.

DONT – While Clone methods are implemented, ICloneable is not implemented or used in any public or private APIs.

2.7.4 IComparable<T> and IEquatable<T>

DO – Object.Equals is overridden when implementing IEquatable<T> in JobDefinition class.

2.7.5 Object

DO - Unit tests in JobDefenitionTests show that overriding of Object.Equals comply with the contract, making x.Equals(x), x.Equals(y), y.Equals(x), x.Equals(z) and so on.

DO – GetHashCode is override when implementing IEquatable in JobDefinition.

CONSIDER – IEquatable<T> is implemented whenever overridden Object.Equals.

DONT – No exceiptions is thrown from Equals in JobDefinition.

DO – GetHashCode is overridden when overridden Object.Equals.

DO – Comparing two objects from a class implementing Object.Equals, as JobDefinition will return true if GetHashCode returns the same value for these objects.

DO – By using GUID to generate JobIds the distribution of numbers should be uniform.

DO – GetHashCode should return exaclty the same value regardless of chances of any valid changes made to an object, because the JobId cannot be changed after initialization. AVOID – No exceptions are thrown from GetHashCode.

DO – ToString is overridden on Job and AdvandedJob to get necessary info about a scheduled job.

DO – String returned from ToString in Job classes does span several lines, but only necessary properties are returned that hold info about triggers and statuses.

CONSIDER – Due to GUID generating JobId, toString will always return a unique string from each object.

DO – JobId might be unreadable to human eyes, but there is other interesting info such as name of the trigger used, JobPriority and JobStatus.

DONT – No empty or null is returned from ToString.

AVOID – No exceptions are thrown from ToString. Because all properties have to be set when exiting Job constructor.

DO – Do not know what could cause a side effect from these properties written out in the ToString method. JobId, TriggerHandler name, JobPriority and JobStatus will always have a value, unless GUID generations can lead to unforeseen bugs.

Solution

3.1 TaskyFy 1.0.0 Release

TaskyFy 1.0.0 was ready for internal release on 31.03.2025 for grading and user testing. The release included a NuGet package to be installed as a ready-to-use library and documentation.

3.1.1 Current Features

Most critical features specified in were implemented as intended, see table 3.1 below comparing original requirements specifications to actual implementation. Features under construction are not included in this table.

Table 3.1: Original requirements vs actual implementation

#	Requirement	Actual implementation	Category
1	Intervals (implemented in TimedTrigger)	Was meant to be its own trigger. Now implemented as an option in the TimedTrigger class.	Core (functional)
2	Calendar class	Was implemented but decided later to keep track of time directly in TimedTrigger class and rather use a priority system described in point XX to handle overlapping Jobs in a given time. May still be used to keep track of Jobs scheduled in different time zones.	Core (Technical)
3	Timed triggers	Custom code can be executed at a specific date, time of day or given time from now. This class remains mostly the same as first intended.	Core (functional)
4	Loosely coupled code architecture		Architecture (technical)
5	Iteration through several processe	This is managed by JobHandler class.	Core (functional)
6	Event triggers	Actions that require tasks to be triggered by specific events. It works by subscribing to events in classes that implement the IPublishJobs interface.	Core (functional)

#	Requirement	Actual implementation	Category
7	Task/model class	Renamed to Job class to not be confused with the Task class in System.Threading.Tasks. It stores all information; trigger type and custom code implemented by the user and serves as the main model class.	Architecture (Technical)
8	Start, stop and pause	The library is able to add, start and stop scheduled jobs using the JobHandler class.	Core (functional)
9	Task manager	Renamed JobHandler, serves as the manager for handling scheduled Jobs. An internal class JobStorage is used to store Jobs and works as a dependency in JobHandler	Architecture (Technical)
10	Trigger interface	A user can implement custom code by using lambda expression as an argument to the Job model class.	Core (Technical)
11*	JobBuilder	Was not in the original requirement specifications. Create Jobs more easily with the JobBuilder class	Functional

Described in point 11 in table 3.1 the team wanted to explore opportunities to instantiate scheduled Jobs more fluid and was looking into either factories or Fluent API. Wanting to learn more about builder pattern and felt more convenient to use so the library ended up with Flutent API (Chapsas, 2021).

3.1.2 Concerns Regarding First Release

3.1.2.1 Missing XML documentation

XML documentation is in the source code, but the team forgot to generate XML documentation in the NuGet build. This may be confusing for participants in the user test to only rely on the documentation and IntelliSense. Will be fixed in release 2.0.0.

3.1.2.2 Public APIs that should be internal

Some APIs were intended to be internal but ended up as public, like ExecuteJob in the Job class. Some might try to schedule a Job using that method directly, but it's intended to be used by JobHandler internally to schedule a job for execution. It should be marked as deprecated in the next release.

3.1.2.3 Chaining methods in JobBuilder

The vision of flexibility in JobBuilder is to achieve chaining of methods in any order. The only exception is not combining methods that define different kinds of triggers and calling BuildJob() at the very end because it returns the newly created Job. Unfortunately, some methods have to be chained in a specific order to function, like defining triggers before description and actions. This may cause some confusion in the test and will be fixed in release 2.0.0. It has been discussed that methods like SetDescription and SetEventTriggerThreshold should be optional in a minimal use case scenario, but now it is mandatory to use. Based on feedback from user tests changes may be reflected in release 2.0.0.

3.1.2.4 Combining timed and event driven Jobs

As of release 1.0.0 it's not possible to schedule time-based Jobs to act as publishers for Event-driven Jobs. The only option is to create custom classes that implement IPublishJob interface to act as publishers for an Event-driven scheduled Job to listen to. This feature was not ready in time for releasing 1.0.0 but will be included in 2.0.0.

3.1.2.5 Cancelling Jobs using CanellactionToken

A StopJob method was implemented to let users be able to stop scheduled jobs from execution. Which is fine, but asynchronous methods do not offer CanellcationTokens, which break some rules in chapter 9.2.2 Task-Based Async Pattern in Microsoft Framework design guidelines. CanellcationTokens will be added in release 2.0.0.

3.1.2.6 Handling Jobs with JobHandler IDs

Inspired by how Linux creates IDs for each process (Baeldung, 2025), the idea of managing scheduled jobs with incremental IDs generated by JobHandler. It seemed like a good design decision, but only if the user knows how to and which Jobs to lookup with IDs. After the initial release it raised multiple concerns that it could make handling Jobs inefficient and confusing for first time users. For instance, finding a scheduled Job requires a lookup and retrieve a list of IDs and description, then the user can call any JobHandler methods with the ID belonging to the desired Job. Based on feedback from user test it may be updated in release 2.0.0.

3.1.2.7 Returning values

Since the Job class takes the Action delegate as an argument, custom implemented methods can only return void, which is limiting different use cases for the user. It would make TaskyFy much more versatile to be able to return values by using Func as a parameter.

3.2 TaskyFy 2.0.0 release

3.2.1 TaskyFy 2.0.0 Release

Taskyfy 2.0.0 was ready for internal release on 27.05.2025 for grading and user testing. The release included a NuGet package, API documentation, and source code. Full documentation is specified in the attachment UserGuide TaskyFy 2.0.0.

3.2.1.1 New features

TaskyFy 2.0.0 has major new features in retry logic in failed tasks, returning values from user provide code and logging in files instead of the console. New improvements include new APIs in JobHandler and better experience when chaining methods in JobBuilder. A bug was fixed in EventTrigger making it not listen to intended. Refer to table 3.2 for details and the attached UserGuide TaskyFy 2.0.0 for full documentation.

Table 3.2: TaskyFy 2.0.0 Release Features

#	Requirement	Implementation	Category
1	Retry logic	Scheduled tasks can automatically attempt retry after initial failure.	New feature
2	Logger	Log messages are written to a txt files in a new folder “Logs” instead of the console.	New feature
3	Returning values	AdvancedJob class can be specified to return values from user provided code	New feature
4	Priority scheduling	Jobs can be scheduled to have	New feature
5	JobHandler	New or improved APIs for managing scheduled tasks. Jobs can be started, stopped and removed by reference as an alternative to storageId.	Improved feature
6	JobBuilder	Some methods are optional to chain like SetDescription. JobBuilder can be used both for common and advanced scenarios.	Improved feature
7	EventTrigger	EventTrigger can listen to other scheduled jobs or all classes implementing IPublishJob interface. A feature that did not work as intended with release 1.0.0	Bug

3.3 UML diagram

The UML diagram is high res and can be zoomed in to view

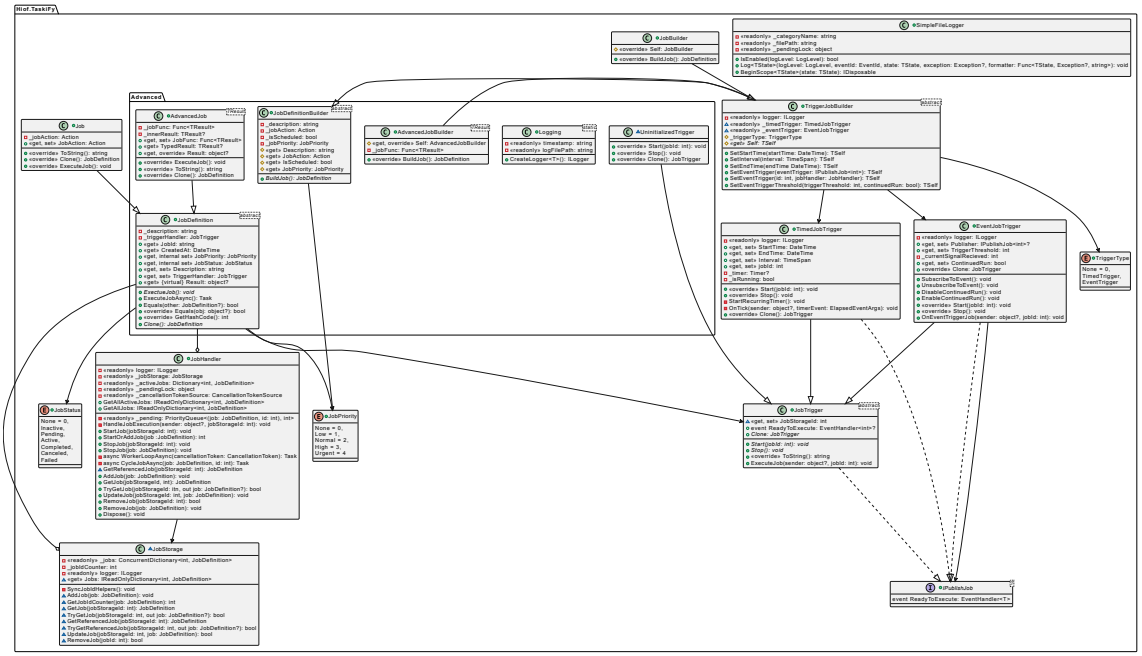

Figure 3.1: UML class diagram

User Test

4.1 Planning

Before we could carry out the user test, the group came together to strategize a setup for the test in a way that would increase our chances of getting constructive feedback, that we could utilize to make further improvements on our framework.

During our meetings we considered different aspects of pros and cons regarding to our solution, what would our framework potentially be used for, and what potential problems would this solution solve? We also to into consideration the shortcomings of our solution, and if the design regarding to naming conventions were logical and easy to understand. Lasty there was a reflection round were meeting participants discussed which aspects of the framework that the group wished to focus on regarding to the test. Here we used the framework for TaskyFy(1.0.0) and it's corresponding User Guide documentation as inspiration for the structure of the user test.

As the meeting progressed, the group came up with a priority list of elements in the framework that should be tested:

1. **Naming Conventions**
    - Does user find the naming description coherent with what the code does?
    - Are there aspects of the code that breaks the rules of .Net naming conventions?
2. **Functionality and Logic**
    - Does the implemented code do what the user intended it to do?
    - Did the user find the solution intuitive and easy to use?
    - Were the aspects of writing the code that user would prefer to be different?
3. **Readability**
    - Was the appended documentation well organised and easy to read?
    - Was the appended documentation coherent with the frameworks code structure?
    - Were there aspects of the documentation that should be formatted in a more readable format?

4.2 Building the Test

The created test was built up with the following components:

- NuGet package for TaskyFy(1.0.0).
- Predefined Test user environment.

- User Guide TaskyFy(1.0.0) document.
- User Test document.
- Test task solution examples document.
- Participant schema MS Forms.

4.2.1 NuGet Package

The Nuget package used for this test was build on the final version of TaskyFy(1.0.0) which has been branched out in our DevOps Repo under 'release/1.0.0'. This is the same NuGet as the one in the appended compressed file for TaskyFy(1.0.0).

4.2.2 Test user environment

By the end of our planning, we had created a predefined Test user environment which we intended the testers to use under the test. This was done to save time during the tests, as the priority of our testing were focused on the framework and its documentation itself, rather than to use time of the test on installation of the framework in a new environment.

The environment is build up on a minimal class library, and a empty CLI file where the user is intended to complete the given tasks.

This is the environment that the testers were introduced to:

TicketSalesPublisher.cs

Listing 4.1: Class Library

```
using System;
using Hiof.TaskyFy;

namespace TicketSaleCLI
{
    internal class TicketSalesPublisher
        : IPublishJob<int>
    {
        public event EventHandler<int>?
            ReadyToExecute;

        public void RegisterSale()
        {
            Console.WriteLine("Payment
                received.");
        }

        public void SendReceiptOnMail()
        {
            Console.WriteLine("Receipt
                sent to customer.");
        }
        public void RegisterSeat()
        {
            Console.WriteLine("Seat has
                been taken.");
        }

        public void SimulateTicketsale()
        {
            Console.WriteLine("Simulating
                ticket sale event...");
            ReadyToExecute?.Invoke(this,
                1);

        }

    }
}
```

Program.cs

Listing 4.2: User CLI

```
using System;
using Hiof.TaskyFy;

namespace TicketSaleCLI
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Input from tester here...
        }
    }
}
```

4.2.3 User Guide TaskyFy(1.0.0)

In regards of the test, then this document was intended to be the single helping-tool for the tester to have available. The group concluded that this should contain sufficient with information about how to solve the given tasks.

This document was then printed out and given to the user a few minutes before the test commenced.

It's the same documentation that was delivered on release 1.0.0 and can be found in the appended compressed file for 'TaskyFy(1.0.0)'.

4.2.4 User Test

This document is created as the main document for completing the user test.
It's built up of two main components:

1. Introduction
2. Test Cases

4.2.4.1 Introduction

1.1 Goal and Test Objective

The goal of this user test is to evaluate and improve the user experience of the TaskyFy Scheduler Library (version 1.0.0) and to validate that its framework functions as intended via the V1.0.0 NuGet package. By guiding participants through a CLI-based scheduling scenarios, we aim to uncover errors, gaps, or misunderstandings early in development so they can be addressed before future releases.

Under the supervision of project-group moderators, participants will work in a pre-configured command line environment using the provided solution—comprising both the NuGet package and its accompanying class library. Each tester will receive five distinct test cases that demonstrate typical automation scenarios. Success is defined as correctly solving at least three of these cases; note that some later cases depend on earlier ones. Throughout the test, moderators will engage in active dialogue with participants, take detailed notes, and solicit feedback on both the technical functionality and overall user experience.

1.2 Test Environment

Tests are carried out in person with the participant and project-group representatives present, involving active discussion between moderator and participant and using a computer provided by the moderator. The tasks are delivered in a pre-configured development environment on the participant's machine (1.7 Prerequisites), where usage documentation and API references are accessible both before and during the test.

1.3 Roles

- Two moderators oversee the test. During the session, one guides and converses with the participant while the other records minutes.
- Up to two participants may work together: one writes code and the other observes. Both may actively engage in discussion with the moderators.

1.4 Conducting the Test

I. **Introduction**
The moderator welcomes the participant and thanks them for taking part. A brief chat follows, introducing the purpose of the test and its procedures.

II. **Familiarization**
The participant will be given up to 5 minutes to review the documentation, which includes a user guide and API references.

III. **Task Execution**

- Tasks are handed out in chronological order (see chapter 2: Test Cases).
- A pre-set computer with the required environment (detailed in section 1.7 Prerequisites) is used.
- The participant completes each task in turn, maintaining an active dialogue with the moderator about their progress.
- The second moderator takes notes under the ongoing test.

IV. **Post-Test Debrief**
Once all tasks are finished, there's a short discussion between the moderator and participant(s), giving them the chance to share feedback on the user experience and offer constructive suggestions for improvement.

1.5 Collection of Test Results

Qualitative data is collected from coding assignments, observations and conversations between moderator and tester throughout the test. Metrics are divided into three groups:

- **Information about the participant (Filled out pre-test)**
  - Profession, such as student or software developer.
  - Knowledge of .NET platform.
  - Participant number (for anonymous statistic data).
- **Test observations**
  - Is JobBuilder used to initialize scheduled tasks in any of the cases.
  - Time it takes to solve each case.
  - Does any cases seems unnecessarily challenging to solve with the given APIs?
  - Participant's solution compared to the expected outcome.
  - Does participants find the right APIs with the use of IntelliSense.
  - Is the user guide actively being used during the test.
- **Post-test questions**
  (Participants are asked questions about choices made during the test, overall experience and have an opportunity to give constructive feedback and suggest improvements. More questions can be asked on the fly based on reflections made by participants during the test.)

- Why did the participant use JobBuilder (or not) during any cases? Was it convenient to use?
- Did participants find the user guide challenging or easy to understand?
- Did any APIs behave differently than expected based on the signature and documentation?
- Did participants find IDs used to control tasks convenient or logical?
- Suggestions for improvement.

1.6 Privacy

The test is anonymous, but each participant must provide information about their profession and general level of knowledge about C# and .NET platform. Both the code and a written report is stored safely in a OneDrive folder distributed by Østfold University College.

1.7 Prerequisites

- Ensure that Visual Studio 2022 (or Visual Code) or later is installed.
- Unpack the appended zip file containing the solution, and install the Hiof.TaskyFy.1.0.0.nupkg NuGet package into the project as instructed in the appended User Guide.
- The solution contains a Program.cs file with the following structure. This the only place the tester is expected to write code.

Listing 4.3: CLI for tester input

```
using System;
using Hiof.TaskyFy;

namespace TicketSaleCLI
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Input from tester here...
        }
    }
}
```

- The solutions class library consist of a single class (TicketSalesPublisher) that contains some simplified methods, which are intended to be used by the tester to complete the user cases in the given test.

Listing 4.4: Predefined class library

```
using System;
using Hiof.TaskyFy;

namespace TicketSaleCLI
{
    internal class TicketSalesPublisher : IPublishJob<int>
```

```
    {
        public event EventHandler<int>? ReadyToExecute;

        public void RegisterSale()
        {
            Console.WriteLine("Payment received.");
        }

        public void SendReceiptOnMail()
        {
            Console.WriteLine("Receipt sent to customer.");
        }
        public void RegisterSeat()
        {
            Console.WriteLine("Seat has been taken.");
        }

        public void SimulateTicketsale()
        {
            Console.WriteLine("Simulating ticket sale event...");
            ReadyToExecute?.Invoke(this, 1);

        }

    }
}
```

4.2.4.2 Test Cases

2.1 Basic Job Scheduling

Goal

Verify that the program can schedule a one-time job.

Steps

- Define a job by using the JobBuilder class (or create a job with a self-made solution) to create a one-time job and fire it of by using the JobHandler class.
- The job must contain a JobAction which fires when the job starts. The tester can choose to use code from the given library or implement their own code here.
- The job shall be set to start after five seconds.

Expected Results

The job runs correctly after five seconds and fires the code implemented in the jobs jobAction.

2.2 Recurring Job Scheduling

Goal

Verify that a recurring job can be scheduled and works properly.

Steps

- Create a recurring job that starts after five seconds and runs every third second after start.
- The job must contain a JobAction which fires when the job starts. The tester can choose to use code from the given library or implement their own code here.

Expected Results

The job executes after five seconds and fires the implemented code every third second after start.

2.3 Event Driven Job

Goal

Verify that trigger functionality for event-driven jobs works as intended.

Steps

- Create a event-driven job that triggers on a method call from a created object from the TicketSalesPublisher class (or create a trigger of your own device).
- The job must contain a JobAction which fires when the job triggers. The tester can choose to use code from the given library or implement their own code here.
- Tester is free to choose for themself if the trigger should be a recurring event or a single triggered event.

Expected Results

The job reacts on the given trigger event and runs the implemented code correctly when triggered.

2.4 Job Handling (Starting and Stopping Jobs)

Goal

Verify that stopping a started job by using JobHandler works as intended.

Steps

- Use the JobHandler to add and start one of the jobs created in earlier task.
- Make sure the started job is not stopped before intended. This can be solved by making the job recurring or halt the end of the program with Console.ReadLine().
- While the job is running, use JobHandler to stop the job before the program terminates.

Expected Results

The job stops successfully at the intended moment without issues.

2.5 Monitor Active Jobs)

Goal

Control that the system displays active jobs in the queue correctly.

Steps

- Use the JobHandler to add and start one of the jobs created in earlier task.
- Make sure the started job is not stopped before intended. This can be solved by making the job recurring or halt the end of the program with Console.ReadLine().
- While the job is running, use JobHandler to ReadActiveList and monitor the active jobs in the queue.

Expected Results

The active jobs in queue are listed correctly.

4.2.5 Solution Examples

This document is only meant for the person observing the tests. This is written with the intention of giving the observer a comparative to the tester solution in according to how we envisioned how the task would be solved. There is no requirement that the tester creates an identical code as written in the solution, as this is more a reference for expected syntactical setup of the task solutions.

4.2.5.1 Solution for Test Case 1

Listing 4.5: Solution for Test Case 1

```
using System;
using Hiof.TaskyFy;
using Hiof.TaskyFy.Advanced;

namespace TicketSaleCLI
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Define the one-time job using JobBuilder.
            JobBuilder oneTimeJobBuilder = new JobBuilder();
            // Create a JobHandler object.
            JobHandler jobHandler = new JobHandler();

            // Create an object from class library.
            TicketSalesPublisher ticketPublisher = new();

            // Create job
            JobDefinition oneTimeJob = oneTimeJobBuilder
                .SetStartTime(DateTime.Now.AddSeconds(5))
                .SetDescription("One-time Job")
                .SetJobAction(() =>
                {
                    // Code to execute when the job runs
                    ticketPublisher.RegisterSale();
                })
                .SetScheduled(true)
                .BuildJob();

            // Add built job to jobHandler
            jobHandler.AddJob(oneTimeJob);
            //Get ID for current job
            int jobId = jobHandler.GetAllJobs.Keys.First();
            // Start job
            jobHandler.StartJob(jobId);

            Console.WriteLine("Job scheduled to run in 5 seconds.");

            // Loop that runs as long as there are active jobs in queue.
            while (jobHandler.GetAllActiveJobs.Count != 0)
            {
                Thread.Sleep(500);
            }
        }
    }
}
```

4.2.5.2 Solution for Test Case 2

Listing 4.6: Solution for Test Case 2

```
using System;
using Hiof.TaskyFy;
using Hiof.TaskyFy.Advanced;

namespace TicketSaleCLI
{
    internal class Program
```

```
    {
        static void Main(string[] args)
        {
            // Define the one-time job using JobBuilder.
            JobBuilder recurringJobBuilder = new JobBuilder();
            // Create a JobHandler object.
            JobHandler jobHandler = new JobHandler();

            // Create job
            JobDefinition recurringJob = recurringJobBuilder
                .SetStartTime(DateTime.Now.AddSeconds(5))
                .SetEndTime(DateTime.Now.AddMinutes(1))
                .SetInterval(TimeSpan.FromSeconds(3))
                .SetDescription("Recurring Job")
                .SetJobAction(() =>
                {
                    // Code to execute when the job runs
                    Console.WriteLine("Code is excecuted!");
                })
                .SetScheduled(true)
                .BuildJob();

            // Add built job to jobHandler
            jobHandler.AddJob(recurringJob);
            //Get ID for current job
            int jobId = jobHandler.GetAllJobs.Keys.First();
            // Start job
            jobHandler.StartJob(jobId);

            Console.WriteLine("Job scheduled to run in 5 seconds.");

            // Loop that runs as long as there are active jobs in queue.
            // Manual check on endtime added.
            // Can also be solve with just "while (true)"
            DateTime jobEndTime = DateTime.Now.AddMinutes(1);
            while (DateTime.Now < jobEndTime || jobHandler.GetAllActiveJobs.Count
                > 0)
            {
                Thread.Sleep(500);
            }
        }
    }
}
```

4.2.5.3 Solution for Test Case 3

Listing 4.7: Solution for Test Case 3

```
using System;
using Hiof.TaskyFy;
using Hiof.TaskyFy.Advanced;

namespace TicketSaleCLI
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Define the one-time job using JobBuilder.
            JobBuilder eventJobBuilder = new JobBuilder();
            // Create a JobHandler object.
            JobHandler jobHandler = new JobHandler();

            // Create an object from class library.
```

```
            TicketSalesPublisher ticketPublisher = new();

            // Create job
            JobDefinition eventJob = eventJobBuilder
                .SetEventTrigger(ticketPublisher)
                .SetEventTriggerThreshold(1, true)
                .SetDescription("Ticket sale")
                .SetJobAction(() =>
                {
                    // Code to execute when the job runs
                    ticketPublisher.RegisterSale();
                    ticketPublisher.RegisterSeat();
                    ticketPublisher.SendReceiptOnMail();
                })
                .SetScheduled(true)
                .BuildJob();

            // Add built job to jobHandler
            jobHandler.AddJob(eventJob);
            //Get ID for current job
            int jobId = jobHandler.GetAllJobs.Keys.First();
            // Start job
            jobHandler.StartJob(jobId);

            int sales = 0;

            while (sales < 5)
            {
                ticketPublisher.SimulateTicketsale();
                Thread.Sleep(1000);
                sales++;
            }
        }
    }
}
```

4.2.5.4 Solution for Test Case 4

Listing 4.8: Solution for Test Case 4

```
using System;
using Hiof.TaskyFy;
using Hiof.TaskyFy.Advanced;

namespace TicketSaleCLI
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Create a JobHandler object.
            JobHandler jobHandler = new();

            int jobCount = 0;

            // Create a job using the builder
            JobDefinition job1 = new JobBuilder()
                .SetStartTime(DateTime.Now.AddSeconds(5))
                .SetInterval(TimeSpan.FromSeconds(3))
                .SetEndTime(DateTime.Now.AddSeconds(60))
                .SetJobAction(() =>
                {
                    jobCount++;
                    Console.WriteLine($"Job runned {jobCount} numer of times.");
```

```
                })
                .SetDescription("Test stopping job 1")
                .BuildJob();

            // Add built job to jobHandler
            jobHandler.AddJob(job1);

            // Start the job using the ID generated by jobHandler
            jobHandler.StartJob(1);

            while (true)
            {
                if (jobCount == 5)
                {
                    jobHandler.StopJob(1);
                    Console.WriteLine("Job 1 stopped.");
                    break;
                }
            }
            Console.ReadLine();
        }
    }
}
```

4.2.5.5 Solution for Test Case 5

Listing 4.9: Solution for Test Case 5

```
using System;
using Hiof.TaskyFy;
using Hiof.TaskyFy.Advanced;

namespace TicketSaleCLI
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // Create a JobHandler object.
            JobHandler jobHandler = new();

            // Create an object from class library .
            TicketSalesPublisher ticketPublisher = new();

            // Create a job using the builder
            JobDefinition oneTimeJob = new JobBuilder()
                .SetStartTime(DateTime.Now.AddSeconds(5))
                .SetJobAction(() =>
                {
                    // Code to execute when the job runs
                    ticketPublisher.SimulateTicketsale();
                    Thread.Sleep(5000);
                })
                .SetDescription("Simulate ticket sale")
                .BuildJob();

            // Add built job to jobHandler
            jobHandler.AddJob(oneTimeJob);

            // Retrieve and loop through added jobs to monitor their status and
                IDs to control the job behavior (optional, but useful to get the
                id).
            foreach (var job in jobHandler.GetAllJobs)
            {
```

```
                Console.WriteLine($"Job ID: {job.Key}, Description:
                    {job.Value.Description}, Status: {job.Value.JobStatus}");
            }

            // Start the job using the ID generated by jobHandler
            jobHandler.StartJob(1);

            // Wait for the job to star execute
            Thread.Sleep(6000);

            // Retrieve all jobs again to monitor change in their status
            foreach (var job in jobHandler.GetAllJobs)
            {
                Console.WriteLine($"Job ID: {job.Key}, Description:
                    {job.Value.Description}, Status: {job.Value.JobStatus}");
            }

            // Pause the console to view output.
            Console.WriteLine("\nPress any key to exit...");
            Console.ReadKey();
        }
    }
}
```

4.3 Conduction of User Test

4.3.1 Test Structure

The tests have been completed through several sessions with a total of six participants.

Table 4.1: List of test groups

Date of test	ID	Profession	Coding experience	Group size
08.05.2025	1	Student	Beginner	2
08.05.2025	2	Student	Beginner	2
09.05.2025	3	DevOps ingeniør	Expert	1
09.05.2025	4	IT konsulent	Good	1

The testers were given the user guide and predefined test environment 5-10 minutes before starting the test. This was done so that the time to read documentation didn't get accounted for when measuring the time taken to complete each task.

Testers were not given a deadline for when to complete the test. Instead, we timed how long each group used on each task.

When the test was finished, the participants were prompted with a prewritten questionnaire where they were given the possibility to give feedback in open writing.

4.4 Results and Findings

When all the test and queries were conducted. The gathered data was combined and analysed by using MS Forms(Direct link to the online form).

This is the following feedback we based further development on:

- **Any struggles to complete the cases?**
  - **Case 1:**
    I. Prøvde å legge til en jobb ved å bruke StartJob og ikke AddJob først i JobHandler, prøvde å bruke jobben som argument i StartJob og ikke sjekke ID som genereres først av JobHandler. Sto litt fast på å legge til en JobAction, var ikke kjent med å bruke anonyme/arrow funksjoner.
    II. Fant ikke ut hvordan starte jobben, prøvde først å legge til referanseobjektet som ble opprettet, men skjønte lite da StartJob() ba om en int som ikke var jobbens genererte id. Ble veiledet av moderator til å skrive inn tallet 1 som id nummer i metoden.
  - **Case 1 og 2:**
    I. Ble forvirret av å måtte sette StartTime før JobDescription og JobAction i JobBuilder
    II. Testen krasjet fordi JobDescription ikke eksplisitt ble brukt i JobBuilder.
  - **Case 3:**
    I. Litt knotete å forstå seg på EventTrigger. Satt først 0 som argument i SetEventThreshold(), fikk aldri feil/tilbakemelding på hvorfor jobben ikke ble kjørt.
    II. Testen krasjet fordi SetDescription ikke ble satt.
    III. Prøvde å simulere jobben flere ganger, men skjedde kun en gang da Threshold ikke ble satt. Sjekket dokumentasjonen og fikk satt opp.
    IV. Alle casene ble fullført, men litt utfordrende ved noen tilfeller - Beskrevet nærmere i avsnitt 8.
    V. Var ikke så kjent med Fluent API fra før så tokk litt tid å sette opp JobBuilder for første gang. Skumleste dokumentasjonen, men hadde ikke sett på eksemplene for hvordan bruke builderen i bibliotekets tilfelle.
    VI. Slet med å finne ut hvordan å administrere jobber som lå i JobHandler, dvs: hvordan få tak i ID som ble generert av JobHandler, siden ikke Job objektet sin ID brukes.
  - **Case 5:**
    I. Brukte en del tid på å sette opp logikk for å gå gjennom jobber som blir hentet med en løkke.
- **General observations.**
  - **Case 1:**
    I. Lagde JobBuilder som egen klasse separat før det ble opprettet en Job.
    II. Opprettet JobBuilder klasse så så JobHandler. Fant ut raskt at det måtte opprettes en Job som ble satt til å være av JobBuilder objektet.

III. Startet med å legge til BuildJob, men fant ut at det er den avsluttende metoden for å opprette en Job. Testen ble gjennomført med riktig utfall.
IV. Satt opp JobBuilder og JobHandler + job onetimejob = jobbuilder, starttid, beskrivelse, jobaction, setscheduled. Forsøkte å bygge jobben, men feilet. Får beskjed om at hvis de har bygd en jobb må de også starte den, i en form for kø. De har da ikke brukt JobHandler.
V. Satt isteden jobhandler(onetimejob).
VI. De lurte på hvor de skulle sette opp triggere. Ble da gitt beskjed om at de har satt opp en trigger de ikke bruker. De endret og forsøkte å kjøre - programmet rakk å kjøre seg ferdig før jobben fikk lov til å starte. Får beskjed om console.readline - for å feks sette inn while loop for å holde liv i programmet. De satt da Console.readline.
VII. Jobben kjørte med riktig utfall! De brukte 21min.
VIII. Forventet at jobben skulle starte ved å bruke JobHandler sin AddJob() metode. Ble fortalt at en annen metode benyttes for å skedulere jobben (StartJob()). Den ble funnet gjennom Intellisense.
IX. Prøvde å starte jobben ved å bruke Job objektet sin ID. Ble fortalt at JobHandler genererte en ID som brukes til å behandle jobber. Deltager fant getAllJob() metoden gjennom Intellisense, og satt opp en foreach løkke og skrev ut ID nummer til konsollen.

- **Case 2:**

I. Finner SetInterval i JobBuilder med en gang. Prøvde å legge til SetDescription før interval, men det går ikke på grunn av rekkefølgen metoder i arvhierarkiet. La ikke til EndTime. Husket metoder og fremgangsmåte i JobHandler fra før. Testen ble gjennomført med riktig utfall.
II. Starter å opprette en ny JobBuilder.
III. Fant SetInterval med en gang.
IV. Testen er gjennomført med forventet resultat.
V. Satt opp jobhandler og jobbuilder.
VI. Job reccuringJobBuilder = recurringJobBuilder
VII. Får beskjed om at de må ha ett eller annet som gjør at programmet ikke stopper før tiden.
VIII. Forsøkte å kjøre, men fikk opp feil - job does not contain a definition for setstarttime.
IX. Får beskjed om at de må starte jobben - litt samme problem som i case 1.
X. Satt jobhandler.addjob(recurringjob)
XI. int jobid = jobhandler.getAlljobs.keys.first()
XII. Får beskjed om at - er reccuringjob det samme som reccuringjobbuilder?
XIII. Endret job reccuringjob = recurringJobbuilder.
XIV. Beskjed - Hva har dere kalt jobbuilderen deres?
XV. De sa ""Den skal være reccuringjobbuilder"" og setter jobbuilder reccuringjobbuilder = new jobbuilder.
XVI. Jobben kjørte med riktig utfall! De brukte 9 min
XVII. Gjenbruk av builder i case 1. Forsto fort hvordan å endre metoder i builderen slik at det ble lagt til intervall. Løste oppgaven fort.

- **Case 3:**

I. Brukte JobBuilder fra tidligere oppgaver, men kalte builder.SetEventTrigger() og SetEventThreshhold() uten å chaine på de andre metodene.
II. Testen var en suksess. Testet først med et signal i threshold, deretter med 3 signaler i Threshold for å se forskjellen.
III. Ser etter en relevante metoder med Intellisense, fant setEventTrigger og la inn et referanseobjekt.
IV. Testen ble gjennomført med forventet resultat.
V. Kopierte koden fra case 2.
VI. Fjernet reccuringJobBuilder = recurringJobBuilder til eventjobbuilder
VII. job eventjob = eventjobbuilder
VIII. Fjernet setstarttime og endret til seteventtrigger(2, jobhandler), seteventtriggerThreshold(1, true).
IX. Fjernet intervaller
X. Satt så opp jobhandler.addjob(eventjob);
XI. De sa ""Ett signal for å trigge jobben. Her står det at triggeren skal være et metodekall. må lage et objekt fra den""
XII. Setter TicketSalesPublisher ticketSalesPublisher = new TicketsalesPublisher(); øverst - sa ""Hvordan bruker vi den"".
XIII. Fikk feil - job with id 2 not found.
XIV. Sa ""Hvor skal jeg sette job id""
XV. Får beskjed om å fjerne jobhandler i Seteventtrigger. Litt defust i dokumentasjonen, må fikses. Feks en metode så lenge hele objektet er triggeren, vil det trigge hver gang du bruker en metode fra det objektet.
XVI. Satt ticketSalesPublisher.RegisterSeat();
XVII. Får beskjed om at programmet ikke får tid til å registre - de må gi programmet nok tid til å starte jobben.
XVIII. Sa ""ta threads"" setter - threads.sleep(1000).
XIX. Endret ticketSalesPublisher.RegisterSeat(); til ticketSalesPublisher.SimulateTicketsale();
XX. Jobben kjørte med riktig utfall! De brukte 10min
XXI. Gjenbrukte / endret builder objektet fra forrige oppgave.
XXII. Ble opphengt i at dokumentasjonens eksempel på eventDrivenJob lyttet til en annen JobHandler ID for å eksekvere sin egen job, men på tidspunktet av release 1.0.0 går det ikke for EventTrigger å lytte til andre skedulerte jobber.
XXIII. Deltager ble gitt hint som at SetEventTrigger() metoden til builderen kan lytte til andre objekter enn kun de som ligger i JobHandler, og at eksempelklassen TicketSalePublisher har en metode som er ferdig satt opp. Det er fordi det ikke er dokumentert enda hvordan å implementere egenlagde metoder som kan lyttes til av EventTriggeren.
XXIV. Deltager fant deretter metoden SimulateTicketSale() i eksempelklassen og la til referanseobjektet som argument i SetEventTrigger(). Satt opp SetEventThreshold(1, true) og fikk gjennomført oppgaven.

- **Case 4:**
  I. StopJob() metoden mangler i API referanse, men deltager fant StopJob() som relevant å bruke gjennom Intellisense.
  II. Jobben ble stoppet og testen er gjennomført med riktig utfall.

III. Fortsetter fra oppgave 2
IV. Testen ble gjennomført med forventet resultat.
V. Kopierte kode fra case 1.
VI. Sa ""Sett threadsleep her også da"" ""Dette er onetime sync, er det noen stopper? Hvis vi bytter til reccuring""
VII. Endret og kopierte kode fra case 2.
VIII. La til thread 1000 for å se om den stopper reccrurring job så den går hvert 3 sek.
IX. Jobben kjørte med riktig utfall! Brukte 3min
X. Prøvde å se i dokumentasjonen, men deltager fikk hint om at metoden (StopJob()) ikke var lagt til i dokumentasjonen enda.
XI. Fant StopJob() metoden gjennom Intellisense og brukte samme ID som tidligere for å stoppe jobben.

- **Case 5:**

I. Fant GetAllActiveJobs() med en gang.
II. Skrev ut hele toString() av hver aktive job.
III. Fasiten spurte om å bruke Job properties for å få ut informasjon, men skrev ut hele objektet som benyttet toString. Ellers ble testen gjennomført riktig."
IV. Starter å lese dokumentasjon for å vanlig bruk.
V. Ser etter relevant metoder i JobBuilder med Intellisense.
VI. Prøver å kjøre StartJob() før AddJob(), fant fort ut at det ikke fungerte og sjekket intellisense og fant raskt AddJob() som så relevant ut.
VII. Forvirrende å ikke bare kunne starte en job med referanse objektet.
VIII. Fikk ikke startet en job, glemte å legge til Description.
IX. Testen er gjennomført.
X. Legger til flere jobber og starter alle med interval. Bruker GetAllActiveJobs i en foreach loop og skriver ut ID og description til alle jobber.
XI. Testen ble gjennomført med forventet resultat."
XII. "Gruppe på 2 studenter som gjennomførte testen. Begge leste og satt seg inn i dokumentasjonen først, så fordelte de seg ved at den ene kodet mens den andre leste dokumentasjonen og guidet.
XIII. Kopierte kode fra case 4.
XIV. De sa ""Bare bruk samme recurring og fjern thread sleep"".
XV. Får beskjed om at det kan være den ikke klarer å finne Getalljobs. Jobben starter ikke før om 5 sek, så vil printe ut tom liste.
XVI. Satt threadsleep(1000)
XVII. Brukte jobklassen, satt foreach
XVIII. Får beskjed om at jobhandler en den som holder på alle jobbene.
XIX. Sa ""get all jobs, print til konsollen""
XX. Får beskjed om at jobbuilder blir definert som key, og description er job.value.description. Må skrives ut som string. Sett som $ tegn. Legg inn job.key i brackets.
XXI. Sliter litt med innhold i foreach. Konverteringsfeil?
XXII. Får beskjed om at programmet forventer job, men dere vet ikke så derfor sett var.
XXIII. Satt var.

XXIV. Jobben kjørte med riktig utfall! De brukte 12min

XXV. Hadde allerede satt opp en foreach løkke til å gå gjennom jobber i JobHandler.

XXVI. Brukte GetAllJobs() og ikke GetAllActiveJobs(), men utgjorde lite forskjell når det var så få jobber i JobHandler."

- **Did the participant use JobBuilder**
  If why, why not? If yes, was it convenient to use?

I. JobBuilder var lett å bruke og navn på metoder ga mening. Deltagere var ikke kjent med Builder klasser fra før og litt usikker på hvordan å komme igang. Synes ikke det var for mye setup.

II. Deltager var kjent med Flutent API fra før. Forvirrende å måtte sette enkelte metoder i en bestemt rekkefølge, f.eks SetDescription og JobAction må komme etter trigger relateterte metoder. SetJobDescription og EventTriggerThreshold burde kunne sløyfes for å få en default verdi.

III. Ja, for å lage en jobb. De sa "synes det var lett å forstå. Er alltid vanskelig i starten, men når man får tid til å sette seg ordentlig i det så er det oversiktlig"

IV. Tok tid å venne seg til å bruke builderen ettersom deltager ikke var så kjent med det. Satt opp builderen så den kunne gjenbrukes i flere job objekter, som virket intuitivt. Enkelte metoder var forvirrende å bruke, som: SetScheduled(), den er dokumentert som at en job skeduleres med en gang den legges til i JobHandler, men den har ingen funksjon. Den ble forsøkt brukt, men gjorde ingenting. Forvirrende å hva som skulle som argument i SetEventTrigger(), siden eksempet i dokumentasjon viste.

- **Did the participant find the user guide challenging or easy to understand?**

I. Må oppdatere dokumentasjon med nyeste metoder, men beskrivelser av APIer var oversiktlig og beskrivende.

II. Savnet at XML dokumentasjonen dukker opp når en holder musepeker over metoden som skrives. Kanskje noe som må settes på i eksport av NuGet pakke? Kunne vært litt bedre struktur av metoder i API referanse, kanskje på tabellformat?

III. De synes user guiden var grei. Slet litt med å skjønne at de måtte koble jobbene. addjob, startjob osv. Sa at det gir mer mening nå, men ikke i starten. Sa at de skulle ønske det var vist en hel implementasjon med addjob, startjob osv. Vise at man må ha noe mer.

IV. Synes dokumentasjon var litt forvirrende.
Noen metoder manglet, som f.eks ReadAllActiveKeys() (i brukertest), StopJob() i Jobhandler og dokumentasjon for å lage egne trigger metoder.
Det kunne vært dokumentert med flere kodeeksempler på bruk av EventTrigger, både med å lytte til en annen skedulert jobb, og en egenlaget med publisher interfacet.

- **Did any APIs behave differently**
  than excepted based on the signature and documentation?

I. Forvekslet AddJob() og StartJob(), men ellers var navn på metoder og oppførsel beskrivende og oversiktlig.

II. Nei.

III. Nei, var beskrevet slik det funker

IV. At AddJob() ikke skedulerte jobben som skulle kjøres, bare la den til. Fant det lite intuitivt å måtte legge til en job, for så å starte den med AddJob().
V. At StartJob() måtte ta imot en generert ID fra JobHandler og ikke kunne benytte Job referanse objektet.

- **Was IDs that controls task behaviour convenient or challenging to use or find?**

I. Var ikke åpenbart at det ble generert en ID for hver job i JobHandler så deltager slet i starten med å legge til og starte en jobb. Men så fort det ble klart gikk det fint med de andre oppgavene.
II. Burde kunne starte jobben med JobId eller bare job objektet. og ikke bare den som jobHandler genererer.
III. OK. Finne foreach løkken - måtte ta som "var"
IV. Ja

- **Suggestions for improvements?**

I. Oppdatere API dokumentasjon med StopJob og at custom metoder som benytter EventTrigger må implementere IPublish Interfacet og opprette en EventHandler som brukes Invoke i aktuelle metoder.
II. Legge inn en sjekk på SetEventThreshold() om det er lagt inn 0 eller et negativt tall, for nå får ikke bruker tilbakemelding på feil input og hvorfor jobben aldri får signal til å kjøre.
III. Oppdatere brukertest så ReadActiveList blir byttet ut med GetAllActiveJobs.
IV. Mulig å behandle jobbber i JobHandler med Job objektet sin ID i tillegg til den som blir generert av JobHandler.
V. Hvis Job objekt som kjører StartJob() ikke ligger i jobHandleren kan den legges til automatisk istedenfor å måtte manuelt kjøre AddJob() først.
VI. Forstår ikke helt hvorfor IPublishJob interfacet må returnere en int og ikke bare være generisk. Burde kunne returnere hva som helst.
VII. Userguide - vise en hel implementasjon, ikke bare hvordan man bygger, men også hvordan man starter. Hadde vært fint å få en hel kodesnipp.
VIII. Bedre dokumentasjon med eksempler for bruk av EventTrigger.
IX. Starte å skedulere jobben ved AddJob(), eller hvertfall ha færrre steg.
X. JobBuilder: Metoder kan chaines i hvilken som helst rekkefølge.

4.5 Test results

4.5.1 Time usage

Disclaimers on the average time used to complete each task. Case one resolved in a scenario to schedule a simple timed task but involved in creating a Job for the first time. The third task resolved in a more advanced situation creating an event driven task. One of the examples regarding event driven tasks was implemented in a newer version of TaskyFy, not release 1.0.0. The team forgot to exclude it from the documentation, making it confusing for some of the participants.

- Case 1: Half of the participants solved each in about 5-8 minutes, while the other half used between 9-12 minutes.
- Case 2: 5 minutes on average.
- Case 3: 10 minutes on average.
- Case 4: 3-5 minutes average.
- Case 5: 5-8 minutes on average.

4.5.2 Overall feedback

The ID system of JobHandler felt inconvenient and ineffective to use, should be communicated better and make handling Jobs easier. The documentation should include complete examples of building and execute Jobs and how to implement and use IPublishJobs in custom classes. JobBuilder methods should be chained independently and have optional arguments with default values.

4.5.3 Observations using JobBuilder

Surprisingly, all participants chose to use JobBuilder to solve almost all cases rather than creating Jobs manually.

Regarding the given time to read documentation, most participants were not used to fluent APIs and did feel intimidated on defining a Job using JobBuilder. Regardless of that, they all wanted to solve the first case using it. One participant was familiar and did set it up really fast, but an already discovered bug concerning the order of chaining methods did make the participant struggle and needed guidance on how to proceed. The second task resolved to scheduling timed Jobs with intervals, either changing the preceding Job or creating a new one familiarize using JobBuilder. It went more smoothly, some participants did find the right method by reading documentation, others used IntelliSense. Participants found the third scenario to be relatively easy to set up. The main struggle was to understand what argument was to be used in SetEventTrigger() method, mostly because of a mistake in an example of the user guide. Some participants also tried setting 0 or a negative number in SetEventTriggerThreshold(), making the application throw an exception.

4.5.4 JobBuilder feedback

Feedback shows that naming conventions in JobBuilder APIs is good, but users should be allowed to chain methods in any order. Some methods like SetDescription() and SetEventTriggerThreshold() feels optional and should have a default value, not to be mandatory in minimal use cases. All participants managed to chain SetScheduled(), which instantly schedules a Job for execution in JobHandler. But it was discovered that it didnt have any effect and did not throw any exceptions.

4.5.5 Observations using JobHandler

Regardless of what is stated in examples, some participants tried to schedule Jobs without using JobHandler, which did not work. One participant excpected the Job to be scheduled only after

using JobBuilder, while the other tried calling ExecuteJob method on the Job itself, which was intended to be internal. That method was found using IntelliSense and the concern is described in chapter 3.1.2. There was also some confusion about which order to add and start scheduling a Job. Some tried to call StartJob method before adding to JobHandler, while others thought using AddJob also would schedule it for execution.

4.5.6 JobHandler feedback

There were two main concerns that got valuable feedack on how to improve the quality of JobHandler. The core functionality of calling Jobs with JobHandler methods can be improved to accept the reference objects as well as incremental IDs. One suggestion would be to add Jobs by calling StartJob if the Jobb is not already existing in JobHandler. Another suggestion is to add and schedule a Job at the same time using AddJob method. This was the intention by SetScheduled method in JobBuilder, which did not work at initial release.

4.5.7 Improving documentation

As expected, some participants got frustrated about missing XML documentation from the 1.0.0 build, which will be included in release 2.0.0. Some participants also had valuable feedback on the documentation. From small issues like spelling errors, suggestions of including more code examples, to bigger issues like missing API descriptions. Two participants would find it useful to include how to implement custom classes that act as publishers for the Event trigger.

4.5.8 Other suggestions

One participant wanted to create and schedule a custom method which returns a value to be used by other types. It is a planned feature to schedule Jobs that can return values using Func delegates, but for now it is only possible to use Actions.

Discussion

5.1 Potential improvements

5.1.1 Managing jobs in JobHandler

If we could start over, it would be to redefine the APIs in JobHandler, which is the main class users should interact with when managing scheduled tasks. How Linux manages processes inspired JobHandler to give every job an identifying ID that was fast and easy to lookup. The problem was that methods like AddJob did not return the storage Id after adding a job to the dictionary of jobs, leaving users confused on how to proceed.

The only way to retrieve a storageId after adding a job was to use GetAllJobs and then iterate a dictionary with a foreach loop, which made the user experience really bad and violating rules in Microsoft guidelines chapter 2 for ensuring that APIs are both intuitive to use and can be successfully used in very basic scenarios. So how did that happen? We thought the idea was really good and simple, but we did not put ourselves in the shoes of users. If these APIs were user tested before release 1.0.0, this flaw in logic would have been known and fixed. But for release 2.0.0 the solution was to introduce new methods and overloads.

5.1.2 Better planning of naming members and parameters

As the library is expanding, it might happen that when needing new methods, properties or parameters something gets named on the fly without proper importance of naming. For instance, we had some constructor parameters that set properties, but they have some inconsistent naming violating a rule in member design guidelines. But when changing the parameter names it starts violating other rules, leaving a dilemma of breaking some rules in either case. This sparks the thoughts of critical thinking and planning for every public member. Concerns around developing a library have been so much time going into planning types and member names, rather than just getting the job done. Developing software with clean and understandable code often leads to overcomplicating the architecture.

5.1.3 Balance between simple and complex architecture

While developing, we started to feel the complexity of files and abstractions exponentially getting bigger, making the library harder to maintain over time. Every little iteration needed to adjust a lot of other members dependent on each other. We thought of many factors in how this occurred.

All group members were working really hard and wanted to be a part of the project, but difference in experience and wanting to cooperate without breaking each other's code, things were abstracted out fast instead of just when we had a good reason to do so. While just making a BookStore library, the level of complexity and abstractions felt much more intense for a scheduler library. Most group

members did not have any prior knowledge of working with C# or asynchronous code to begin with, making the team unsure how to progress. With little time to develop and deliver the first release, no one felt they had the time to fully test and understand how to schedule tasks. It was all about making it work, trying to balance the use of guidelines, and learning how to code in a C# manner. If we could start over, the first idea would be to fully make task scheduling work in a more minimal setup and expanding over time rather than starting with complex architecture.

5.2 Further development

5.2.1 Persistent storage

Taking backup of active jobs in the system was planned to different file formats such as Json, CSV and XML. By creating an inteface, users should be able to create their own adapter for exporting jobs to other formats or databases. While developing, this feature had many comlications and had to be discontinued due to time constraints and bad planning. Some challenges included serializing triggers with inheritance and polymorphy, having to create a custom json converter to check which trigger was used and which properties to include. The biggest challenge was serializing user provided code and how to deserialize back to Job objects. We didn't understand the huge security risks of exporting and importing code that might lead to a backdoor for malicious programs. We also did not know that code in lambdas was tricky to handle. We worked at several workarounds like only exporting calls to methods, making users only have method names in jobs rather than whole logic. Exporting to CSV or plain text was an easier task, but due to our current inheritance architecture and trigger objects inside job objects it is hard to tackle all usage scenarios. Better structural planning could make persistent storage easier to develop.

5.2.2 Cron expressions on time based jobs

Handling time-based jobs with more accuracy, letting users pass cron-expressions as arguments was under development but discontinued because of time constraints. Learning cron expression patterns was fun and challenging. It could potentially be a powerful feature to set precise time-based tasks at intervals. But since there was no allowance for third party packages, this feature needed a simplified compiler to parse all kinds of expressions to be understood by TimedJobTrigger.

Meeting minutes from supervision

6.1 Meeting minutes 1

Dato og sted:

- Dato: 10.03.2025
- Tid: 08:15–08:45
- Sted: Høgskolen i Østfold, Marius Geitle sitt kontor

Oppmøte:

- Andreas
- Marius
- Kjell-Magne
- Arshad

Forventninger: Faglærer er fornøyd med arbeidet som ble levert inn i forkant av veiledningen.

Prosjektvalg:

- Fakturasystem ble vurdert som en applikasjon og for spesifikt.
- Bordreservasjon ble ikke vurdert som hensiktsmessig.
- Endelig valg ble et mer generelt rammeverk kalt *General TaskManager*.

Diskusjon og spørsmål:

- Arshad: Er det forventet å kunne inspisere kildekoden til rammeverket og se dokumentasjon?
- Svar: Ja. Det skal være mulig å inspisere og dekompilere koden, samt vise XML-kommentarer. I tillegg skal det lages en dokumentasjonsfil (PDF). Den skal være grundig, men ikke for teknisk eller for enkel.
- Marius: Kan vi bruke threading? Vi har sett på Quartz.

- Svar: Ikke anbefalt. Unngå threading. Quartz er for avansert i denne sammenheng.
- Kjell-Magne: Hvor ofte bør man sitere regler fra boka i dokumentasjonen?
- Svar: Det er fint å sitere direkte fra boka. Regler kan brukes for å drøfte designvalg. Det kommer an på strukturen dere velger.
- Arshad: Det kan føre til mange referanser?
- Svar: Ja, men i LaTeX finnes det kommandoer som gjør det mulig å samle referanser til boka i litteraturlista.
- Andreas: Hvor generell bør målgruppen være?
- Svar: Ikke tenk på en spesifikk sluttbruker. Et fakturasystem er for spesifikt – tenk bredere og mer generelt.

Videre arbeid:

- Gå systematisk gjennom ulike scenarioer for bruk av scheduler.
- Kartlegg behov og krav til intervaller i forskjellige apper.
- Eksperimenter med klassestruktur og vurder hva som er realistisk å få til.

Ting å tenke over:

- Hvilke typer oppgaver skal rammeverket kunne kjøre?
- Hvordan løser man konflikter når en jobb kjører ofte, og en annen tar lang tid? (Kø?)

Generelt fokus:

- Fokuser på rammeverk, ikke applikasjon.
- Lag et generelt system som kan brukes i mange domener.
- Vektlegg fleksibilitet og gjenbruk.

6.2 Meeting minutes 2

Dato og sted

- **Dato:** 24.03.25
- **Tid:** 8:15 – 8:45
- **Sted:** Faglærer sitt kontor

Oppmøte

- Andreas
- Hedda
- Kjell-Magne
- Marius
- Arshad

Develop branch

Gruppa har fått beskjed om å ikke bruke GitFlow med en egen `develop`-branch. Går derfor over til å bruke feature branching direkte fra `main` fremover.

Scenarioer og unit tester

- Må få gjort mer unit testing i testverktøyet og ikke scenarioer.
- Skal bruke scenariodrevne unit tester fremfor TDD.
- Det som er skrevet i CLI burde ligge i brukermanualen.
- Skrive forskjellige tester for enkel og avansert bruk.
- Lese koden høyt for å evaluere lesbarhet.
- Skal gjennomføre brukertest av Release 1 med medstudenter.

Sluttlevering av prosjekt

Datoen som står på Studentweb er korrekt for eksamensprosjektet.

Veiledning 21. april

21. april er en rød dag, så veiledningen må flyttes.

Diskusjon rundt kodebasen

- Må fikse `.sln`-filen og flytte den én mappe opp.
- Endre filbaner slik at de samsvarer.

Spørsmål

- Multithreading bør ikke være prioritet.
- Det er greit å bruke `async`.
- `async` bruker kun én tråd, men frigjør CPU ved idle.
- Bruk `SemaphoreSlim` i stedet for `lock`, fordi `lock` ikke er kompatibel med `await`.

Reflections on previous Workshops

The workshops, though not always providing immediate answers, offered valuable insights, shaping our understanding of effective software development.

7.1 Code Quality and Error Handling

We quickly learned the impact of descriptive method naming on code comprehension. This clarity extended to error management: informative and actionable exception messages proved crucial for debugging. We learned that exceptions should replace error codes and that it's critical not to "swallow" exceptions. Preferring built-in exceptions also emerged as best practice. The importance of XML documentation became clear for both code intent and exception details. Overall, we came to understand that clear description is fundamental to all aspects of well written code.

7.2 Design Principles and Structure

The workshops expanded our understanding of design principles. Through the different scenarios, we gained a nuanced view of different design choices, recognizing their specific strengths and implications. We learned the importance of consistent style for readability, realizing how even minor deviations could hinder comprehension.. We refined our use of interfaces versus abstract classes, understanding their specific applications in structuring code.. With experience, we became better at identifying design rule violations and detecting errors more efficently, underscoring the need for well-structured code.

7.3 User Experience

Another reflection was the gap between developer intent and user experience. We observed firsthand that code clear to its creator might be confusing to others, and a framework's purpose isn't always immediately obvious to those who use it. This highlighted the importance of user testing, as a developers code's purpose isn't always clear to users. This taught us to avoid assuming user knowledge or getting blinded by our own work.

Reflection on Previous Coding Assignments

After the completions of coding assignments and the project, the group got together to reflect on our work process and how we would approach similar coding challenges differently with the experience we now possess.

8.1 Planning

A key point that has emerged is the fundamental importance of thorough planning before the actual coding begins. We now see that investing time in outlining the structure, such as:

- Through class diagrams.
- Discussing the solution design beforehand.
- Requirement specification.

If well preformed, then these topics can save considerable time and avoid unnecessary complexity later on.

8.2 Abstraction

When it comes to abstraction, we have learned that it is a balancing act. It can be advantageous to start with a simpler structure and fewer files, then abstract out functionality and components as the need arises and the system grows. In some cases, we experienced challenges in introducing necessary abstraction afterwards, which underscores the value of continuously considering this. At the same time, it is important not to overcomplicate unnecessarily, neither in code nor in folder structure, especially in the early stages.

8.3 Afterthought

Although we feel that we largely followed the same fundamental principles throughout the period, such as using interfaces and a deliberate folder structure, practical experience has given us a deeper understanding of how and when these principles are best applied to create robust and maintainable code. We also take with us the importance of interpreting and applying feedback, even if it may seem vague, to continuously improve our work.

In summary, the practical experience has reinforced the importance of good preparation and an iterative approach to the code's structure and level of abstraction.

Timelister

Table 9.1: Timeliste for Andreas

Andreas Isaksen	Totalt antall arbeidstimer			
	166,55			
Dato(er)	Antall arbeidstimer	Komentarer	Tagg	Pull request nummer
03.03.2025	2,00	Oppstartsmøte	Møte	
04.03.2025	0,30	Laget timeliste	Annet	
07.03.2025	2,00	Møte	Møte	
10.03.2025	1,00	Veiledning	Veiledning	
10.03.2025	3,00	Jobbet med prosjektbeskrivelse	Møte	
14.03.2025	2,00	Møte, kravspekk	Møte	
17.03.2025	3,00	Møte, Ferdigstille kravspekk	Møte	
17.03.2025	3,50	Møte, Features, kanban, Git, Fordele oppgaver	Møte	
17.03.2025	0,50	Opprete modell klasse, git branch "modelClass"	Programmering	
21.03.2025	2,75	Oppsummering og klargjøring for veiledning	Møte	
23.03.2025	2,00	Sette meg inn i eksisterende rammeverk	Annet	
23.03.2025	4,00	Lage useCase løsninger i program.cs	Programmering	
23.03.2025	2,00	Dokument for veiledning 2	Raport	
24.03.2025	1,00	Veiledning 2	Møte	
24.03.2025	2,00	Møte, lage rapport på veiledning, branch merge	Møte	
24.03.2025	2,00	Planlegning av control flow og fordeling av oppgaver	Møte	
25.03.2025	2,00	Modify Job Class	Programmering	
25.03.2025	6,00	Refactor code for Func deligations	Programmering	
28.03.2025	2,50	Møte. Siste innspurt for V.1 -1	Møte	
29.03.2025	5,00	Korrigert/laget XML dokumentasjon for kode i master	Programmering	39
30.03.2025	3,00	Delta på feilretting av kode	Programmering	
30.03.2025	2,00	Møte. Siste innspurt for V.1 -2	Møte	
30.03.2025	2,00	XML kommentar og test for TimedJobTrigger klasse	Programmering	51
31.03.2025	3,00	Avsluttende gjennomgang av XML kommentarer og kontroll av guidelines krav	Programmering	56
31.03.2025	6,00	V.1 final release user guide	Møte	
02.04.2025	4,50	Forberedelse til fremvisning	Møte	
04.04.2025	2,50	Forberedelse til fremvisning	Møte	
04.04.2025	2,00	Forbrede kode til showcase	Programmering	
04.04.2025	2,00	Lage slides til showcase	Annet	
25.04.2025	4,00	Prioritetskø, Func deligations, cancelation token, pendingLocks	Programmering	74
26.04.2025	5,00	Prioritetskø, Func deligations, cancelation token, pendingLocks	Programmering	74
27.04.2025	5,00	Prioritetskø, Func deligations, cancelation token, pendingLocks	Programmering	74
28.04.2025	2,00	Oppsummerings møte	Møte	
28.04.2025	3,00	Lage brukertest	Raport	
29.04.2025	5,00	Lage brukertest	Programmering	
30.04.2025	6,00	Lage brukertest og løsninger	Programmering	
02.05.2025	2,00	Planleggning for brukertest	Møte	
02.05.2025	4,00	Ferdigstille brukertest	Raport	
03.05.2025	4,50	Videre utvikling av AdvancedJob klasse	Programmering	75
04.05.2025	4,00	Avluttende arbeid på AdvancedJob løsning med XML dokumentasjon.	Programmering	75
05.05.2025	1,50	Brukertest møte	Møte	
23.05.2025	2,50	Planlegning for innlevering	Møte	
24.05.2025	6,00	Feilsøking og retting av eksisterende kode	Programmering	
24.05.2025	6,00	UGV2 og tilbakemldinger fra brukertester	Raport	
25.05.2025	6,00	Kartlegge og oppdatere API referanser I UGV2	Raport	
25.05.2025	3,00	Refleksjon workshop/assignments	Møte	
25.05.2025	4,00	Ny eksempelkode og avsluttende arbeid på UGV2	Raport	
26.05.2025	9,00	Rapport bruker test	Raport	
26.05.2025	5,00	Oppdatering av UserGuide og innføring I rapport	Raport	
27.05.2025	3,50	Diverse bistand til avsluttende arbeid	Raport	

Table 9.2: Oversikt timer - Andreas

Oppgave Tag	Antall oppføringer	Timer arbeid med oppgave	%
Programmering	7	29	31,7
Raport	3	6	6,6
Møte	19	45	49,2
Database	0	0	0,0
DevOps	0	0	0,0
Annet	0	0	0,0
Prosjektstyring	0	0	0,0
Veiledning	2	1,5	1,6
Design	1	2	2,18579235
Mars	16	39,5	43,2
April	5	15	16,4
Mai	11	0	0,0
Juni	0	0	0,0

Table 9.3: Timeliste for Kjell-Magne

Kjell-Magne Larsen	Totalt antall arbeidstimer			
	150,00			
Dato(er)	Antall arbeidstimer	Komentarer	Tagg	Pull request nummer
03.03.2025	2,00	Oppstartsmøte	Møte	
07.03.2025	1,50	Prosjektplanlegging	Møte	
08.03.2025	1,50	Utarbeide forslag til prosjekt.	Raport	
10.03.2025	1,00	Veiledning	Veiledning	
10.03.2025	3,00	Jobbet med prosjektbeskrivelse	Møte	
13.03.2025	3,00	Sette seg inn i og oppsett av Azure DevOps (Wiki, boards, repo, instillinger, CI)	Prosjektstyring	
14.03.2025	2,00	Møte om kravspesifikasjon og gå gjennom DevOps i plenum	Møte	
16.03.2025	2,00	Skrive/definere kravspesifikasjoner	Prosjektstyring	
17.03.2025	3,00	Møte, ferdigstille kravspesifikasjoner	Møte	
17.03.2025	3,50	Møte, Features, kanban, Git, Fordele oppgaver, bugfixing i DevOps	Prosjektstyring	3
19.03.2025	3,00	Base trigger klasse, bli kjent med Builder/Fluent API design pattern	Programmering	4
21.03.2025	1,50	Starte på UML klassediagram	Annet	
21.03.2025	2,50	Møte vidreføre scenarioer, fremgangs rapportering og klargjøring til veiledning mandag	Møte	
22.03.2025	6,00	Programmering: Job klasse, JobBuilder, Triggere, Enums	Programmering	13, 14, 18
23.03.2025	5,00	Programmering: JobManager, funksjonalitet og kommentarer. Lese på ConcurrentDictionary, delegates og lambda i C#.	Programmering	
24.03.2025	0,50	Veiledning 2	Møte	
24.03.2025	3,00	Møte, post brief fra veiledning, rydde opp i repos, diskutere veien videre.	Møte	27, 28, 29, 30
27.03.2025	4,00	Refaktor og abstraher Job klasse til JobDefenition, skille mellom JobDefenition (base), Job(enkel), AdvancedJob(avansert/mindre generell bruk). Refaktor logikk i JobHandler og JobStorage.		31, 33, 34
28.03.2025	2,50	Møte. Siste innspurt for V.1 -1	Møte	
29.03.2025	8,00	Endring i access modifiers, navnendringer, slette isScheduled, av ubrukte metoder. La til Clone metoder. Overload konstruktører i AdvancedJob.	Programmering	37, 38, 41
30.03.2025	10,00	Integrere triggere med JobHandler og JobStorage, XML dokumentasjon i JobHandler, navnendringer ac triggere	Programmering	44, 45, 50
31.03.2025	5,00	Sluttspurt av Release 1.0.0, Justert TimeJobTrigger til å kjøre interval for evig uten stopp tid. Unit tests for JobHandler. Endret UninitialTrigger sin access modifier	Programmering	63, 65
31.03.2025	3,00	Oppdatert Clone metoder for å gjennomføre deep copies av Job objeker. Oppdatert XML dokumentasjon, la til kilder. Fjernet Try/catch block i Handle JobAsync	Programmering	67, 69
02.04.2025	2,00	Forberede til fremføring. Fordele oppgaver, sette opp presentasjon og begynne på manus	Annet	
04.04.2025	2,00	Forberede fremføring, øve på presentasjon og utarbeide slides.	Annet	
06.04.2025	2,00	Forberede fremføring og ferdigstille slides. Generalprøve	Møte	
28.04.2025	2,00	Oppsummerings møte	Møte	
02.05.2025	2,00	Planleggning for brukertest	Møte	
03.05.2025	4,00	Oppgaver for brukertest. Skrive om introduksjonsdel og oppgaver til brukertest. Planlegge og sette opp skjema for spørsmål og diskusjon	Annet	
05.05.2025	2,00	Brukertest møte	Møte	
08.05.2025	3,00	gjennomføring brukertest	Annet	
15.05.2025	2,00	gjennomføring brukertest	Annet	
19.05.2025	3,00	Skrive avhandling fra gjennomførte brukertester og konklusjon/diskusjon/utfordringer med Release 1.0.0	Raport	
20.05.2025	3,00	Jobbe med persistent lagring	Programmering	
23.05.2025	2,50	Møte, planlegge siste innspurt. Oppgavefordeling, rapportstruktur	Møte	
24.05.2025	6,00	Fikset bedre chaning av metoder i JobBuilder. Oppdatert JobHandler APIer som samsvarer med brukertester. Prioritetskøen kjørte jobber synkront, fikset slik at jobber kjører asynkront igjen.	Programmering	81, 80, 79, 76
24.05.2025	6,00	Bisto med å fikse event trigger ID som lyttet til feil id når den er subscribet til klasser med IPublishJob interfacet. Jobbe med persistent lagring	Programmering	81, 80, 79, 76
25.05.2025	2,00	Møte gå gjennom workshops og oblig refleksjoner	Møte	
25.05.2025	8,00	Skrive om regler i kapittel 6, 7 og 8	Raport	
25.05.2025	3,00	Endringer i Exception og logging meldinger for å tilfredstille regler i Exception kapittelet. Jobbe med persistent lagring. Det ble desverre ikke noe av.	Programmering	82
25.05.2025	5,00	Bedre eksport/import av eksisterende jobber i JobHandler. Unit tests på JobHandler og Equals/Hashcodes	Programmering	83
26.05.2025	5,00	Bistå med implementering og testing av Retry-logic. Mindre endringer i logger klasser,	Programmering	84
26.05.2025	5,00	Skrive om alle regler som er fulgt i kapittel 2, 6 og litt endringer i 7 og 8. Diskusjon/refleksjon	Raport	
27.05.2025	1,00	Lage NuGet pakke, zipping og ferdigstilling av bibliotek	Prosjektstyring	
27.05.2025	3,00	Ferdigstille rapport, skrive om diskusjon, løsning på release 2.0.0 og konklusjon. Levere prosjekt	Raport	

Table 9.4: Oversikt for Kjell-Magne

Oppgave Tag	Antall oppføringer	Timer arbeid med oppgave	%
Programmering	13	68	45,3
Raport	5	8	5,3
Møte	15	32,5	21,7
Database	0	0	0,0
DevOps	0	0	0,0
Annet	6	14,5	9,7
Prosjektstyring	4	9,5	6,3
Veiledning	1	1	0,7
Design	0	0	0
Mars	23	76,5	51,0
April	4	8	5,3
Mai	18	65,5	43,7
Juni	0	0	0,0

Table 9.5: Timeliste for Arshad

Arshad Abba	Totalt antall arbeidstimer			
	91,50			
Dato(er)	Antall arbeidstimer	Komentarer	Tagg	Pull request nummer
03.03.2025	2,00	Oppstartsmøte	Møte	
07.03.2025	1,50	Prosjektplanlegging	Møte	
08.03.2025	1,00	Lage rapport skjelett	Raport	
10.03.2025	3,00	Prosjektbeskrivelse	Møte	
10.03.2025	1,00	Møte med Geitle	Veiledning	
12.03.2025	1,00	Møte med nytt gruppemedlem	Møte	
17.03.2025	3,00	Møte kravspesifikasjon	Møte	
17.03.2025	3,00	Fordelig av arbeidsoppgaver	Møte	
21.03.2025	2,50	Møte forberede veilending	Møte	
23.03.2025	2,00	UML diagrammer	Design	
24.03.2025	0,50	Møte, veiledning	Veiledning	
24.03.2025	3,00	Møte, post brief fra veiledning, rydde opp i repos, diskutere veien videre.	Møte	
24.03.2025	1,50	Møte, avklare arbeidsoppgaver for uken	Møte	
25.03.2025	7,00	Programmering, TimedTrigger	Programmering	
26.03.2025	5,00	Programmering, TimedTrigger	Programmering	
28.03.2025	2,50	Møte. Siste innspurt for V.1 -1	Møte	
02.04.2025	4,50	Forberedelse til fremvisning	Møte	
04.04.2025	2,50	Forberedelse til fremvisning	Møte	
06.04.2025	1,00	Forberedelse til fremvisning	Møte	
28.04.2025	2,00	Oppsummerings møte	Møte	
30.04.2025	5,00	Logging i programmet	Programmering	
01.05.2025	4,00	Logging i programmet	Programmering	
02.05.2025	2,00	Planleggning for brukertest	Møte	
05.05.2025	2,00	Brukertest møte	Møte	
08.05.2025	3,00	Gjennomføre brukertest	Møte	
11.05.2025	3,00	Logging i programmet	Programmering	
20.05.2025	3,00	Loggin i programmet	Programmering	
23.05.2025	2,00	Møte	Møte	
24.05.2025	2,00	Kode	Programmering	78
25.05.2025	4,00	Skrive i rapport	Raport	
25.05.2025	3,00	Møte	Møte	
26.05.2025	9,00	Skrive i rapport	Raport	

Table 9.6: Oversikt for Arshad

Oppgave Tag	Antall oppføringer	Timer arbeid med oppgave	%
Programmering	7	29	31,7
Raport	3	6	6,6
Møte	19	45	49,2
Database	0	0	0,0
DevOps	0	0	0,0
Annet	0	0	0,0
Prosjektstyring	0	0	0,0
Veiledning	2	1,5	1,6
Design	1	2	2,18579235
Mars	16	39,5	43,2
April	5	15	16,4
Mai	11	0	0,0
Juni	0	0	0,0

Table 9.7: Timeliste for Marius

Marius Rørmark	Totalt antall arbeidstimer			
	103,00			
Dato(er)	Antall arbeidstimer	Komentarer	Tagg	Pull request nummer
03.03.2025	2,00	Oppstartsmøte	Møte	
10.03.2025	1,00	Veiledning	Veiledning	
10.03.2025	2,00	Møte definere scheduler	Møte	
10.03.2025	0,25	Sette opp CI mens vi venter på kjøretid	DevOps	
14.03.2025	2,00	Møte kravspesifikasjon	Møte	
14.03.2025	0,50	Merging and setting up policies on CI PR:!2	DevOps	2
17.03.2025	3,00	Møte ferdigføring av kravspesifikasjon	Møte	
17.03.2025	3,75	Fordeling av arbeidsoppgaver og bugfixing azureDevOps ssh	Møte	
21.03.2025	1,50	Arbeid med å implementere EventTrigger klassen	Programmering	
21.03.2025	2,50	Møte vidreføre scenarioer, fremgangs rapportering og klargjøring til veiledning mandag	Møte	
22.03.2025	4,50	Added events to manage when EventJob is triggered	Programmering	
24.03.2025	0,50	Veiledning	Veiledning	
24.03.2025	3,00	Møte	Møte	
24.03.2025	1,00	Endring av event trigger publisher og navngivning	Programmering	
24.03.2025	2,75	Møte	Møte	
24.03.2025	0,50	Laging av visuelle tegninger for løsning rundt events	Design	
24.03.2025	0,50	Added Xml comments for EventJob og EventPublisher	Programmering	42
27.03.2025	4,00	Changed EventJob to match changes to other parts of the framework	Programmering	42
28.03.2025	2,50	Møte	Møte	
29.03.2025	4,00	Added tests for EventJobTrigger	Programmering	42
30.03.2025	1,00	Changes to names and small optimalization to EventJobTrigger	Programmering	42
30.03.2025	2,00	Finalizing changes to EventJobTrigger for merging	Programmering	42
30.03.2025	2,00	Renaming of all classes derived from trigger and trigger itself	Programmering	44
30.03.2025	1,00	added stop methods for trigger classes	Programmering	52
30.03.2025	2,00	Møte	Møte	
31.03.2025	4,00	Updated JobBuilder classes with new code	Programmering	55
31.03.2025	0,50	added xml comments to JobBuilder classes and removed obsolete classes	Programmering	57
31.03.2025	0,50	Gjennomgang av dokumenter og filer for innlevering	Raport	
10.04.2025	3,00	Forberedelse fremføring	Annet	
05.05.2025	2,00	Brukertest møte	Møte	
08.05.2025	3,00	gjennomføring brukertest	Møte	
20.05.2025	4,00	Reasearch cron expression og base implementasjon	Programmering	
23.05.2025	1,00	Mini møte diskusjon eventTrigger problematikk	Møte	
23.05.2025	3,25	Møte for planlegging av siste inspurt med fordeling av resterende oppgaver	Møte	
24.05.2025	3,00	videre jobb med cron expressions og fixing av feil jobstorageid brukt i eventtrigger	Programmering	76
25.05.2025	4,00	Rapport skriving metode	Raport	
25.05.2025	6,00	Rapport skriving metode	Raport	
25.05.2025	2,00	Møte ferdigstille workshop refleksjon	Møte	
25.05.2025	2,00	møte refleksjoner coding assignments	Møte	
26.05.2025	4,00	gjennomgang kodeforbedring og rapportskriving metode	Raport	
26.05.2025	7,00	Rapport skriving metode og guidlines	Raport	
27.05.2025	1,00	fixing av små feil	Programmering	86
27.05.2025	3,00	Lage classediagram og annet til rapport	Annet	

Table 9.8: Oversikt for Marius

Oppgave Tag	Antall oppføringer	Timer arbeid med oppgave	%
Programmering	15	34	33,0
Raport	5	19	18,4
Møte	16	38,75	37,6
Database	0	0	0,0
DevOps	2	0,75	0,7
Annet	2	6	5,8
Prosjektstyring	0	0	0,0
Veiledning	2	1,5	1,5
Design	1	0,5	0,485436893
Mars	28	54,75	53,2
April	1	3	2,9
Mai	14	45,25	43,9
Juni	0	0	0,0

Table 9.9: Timeliste for Hedda

Hedda Ørnelund Nilsen	Totalt antall arbeidstimer			
	94,40			
Dato(er)	Antall arbeidstimer	Komentarer	Tagg	Pull request nummer
14.03.2025	2,00	Møte kravspesifikasjon	Møte	
17.03.2025	3,00	Møte ferdigstille kravspesifikasjon	Møte	
17.03.2025	3,40	Møte, Git, AzureDevOps, fordele oppgaver	Møte	
19.03.2025	3,30	Programmering, implementere CalendarSystem	Programmering	23
21.03.2025	2,75	Møte vidreføre scenarioer, fremgangs rapportering og klargjøring til veiledning mandag	Møte	
24.03.2025	0,50	Møte, veiledning	Veiledning	
24.03.2025	3,00	Møte, post brief fra veiledning, rydde opp i repos, diskutere veien videre.	Møte	
24.03.2025	2,75	Møte, diskutere veien videre, fordele arbeidsoppgaver	Møte	
25.03.2025	5,50	Programmering, JobHandler og JobStorage	Programmering	33, 45, 50
26.03.2025	7,00	Programmering, JobHandler og JobStorage	Programmering	33, 45, 50
28.03.2025	2,40	Møte, siste innspurt før V.1	Møte	
30.03.2025	2,00	Møte. siste innspurt for V.1 -2	Møte	
30.03.2025	1,00	NuGet test, test lage og implementere pakke i nytt prosjekt	Annet	
30.03.2025	1,00	Programmering, stop metode i jobhandler - ikke implementert	Programmering	
31.03.2025	2,00	Møte, siste innspurt for V.1-3	Møte	
02.04.2025	4,50	Forberedelse til fremvisning	Møte	
04.04.2025	2,50	Forberedelse til fremvisning	Møte	
04.04.2025	1,00	Lage slides til fremvisning	Annet	
06.04.2025	1,00	Forberedelse til fremvisning	Møte	
08.04.2025	0,50	Møte, planlegge for V.2	Møte	
28.04.2025	2,00	Oppsummeringsmøte	Møte	
02.05.2025	2,00	Planlegging for brukertest	Møte	
05.05.2025	1,50	Møte, brukertest	Møte	
08.05.2025	3,00	Gjennomført brukertest	Møte	
23.05.2025	3,00	Møte, planlegge for siste innspurt	Møte	
23.05.2025	4,00	Retry logikk, custom exception	Programmering	87
24.05.2025	2,40	Rapport skriving - metode	Raport	
25.05.2025	3,00	Rapport skriving - metode	Raport	
25.05.2025	6,00	Retry logikk, custom exception	Programmering	87
25.05.2025	3,00	Refleksjon workshop/assignment	Møte	
26.05.2025	3,00	Workshop rapport skriving	Raport	
26.05.2025	2,00	Retry logikk	Programmering	87
26.05.2025	3,00	Custom exception	Programmering	87
26.05.2025	2,30	Testklasse JobRetryLogicTests	Programmering	87
26.05.2025	1,00	Assignment rapport skriving	Raport	
27.05.2025	2,10	Implementere retry logikk, custom exception, unit test	Programmering	87

Table 9.10: Oversikt for Hedda

Oppgave Tag	Antall oppføringer	Timer arbeid med oppgave	%
Programmering	15	34	33,0
Raport	5	19	18,4
Møte	16	38,75	37,6
Database	0	0	0,0
DevOps	2	0,75	0,7
Annet	2	6	5,8
Prosjektstyring	0	0	0,0
Veiledning	2	1,5	1,5
Design	1	0,5	0,485436893
Mars	28	54,75	53,2
April	1	3	2,9
Mai	14	45,25	43,9
Juni	0	0	0,0

References

Baeldung. (2025). *How are linux pids generated*. https://www.baeldung.com/linux/process-id

Chapsas, N. (2021). *How to create your own fluent ap*. https://www.youtube.com/watch?v=1JAdZul-aRQ&t=110s

Cwalina, K., & Abrams, B. (2009). *Framework design guidelines: Conventions, idioms, and patterns for reusable .net libraries*. Addison-Wesley. https://books.google.no/books?id=Jrn5mAEACAAJ

Microsoft. (2025a). *Dictionary<tkey,tvalue>.trygetvalue(tkey, tvalue)*. https://learn.microsoft.com/e%20us/dotnet/api/system.collections.generic.dictionary-2.trygetvalue?view=net-9.0

Microsoft. (2025b). *Ireadonlydictionary<tkey,tvalue> interface*. a%20https://learn.microsoft.com/en-%20us/dotnet/api/system.collections.generic.ireadonlydictionary-2?view=net-9.0